

Automating Continuous Compliance in DevSecOps Through Zero Trust

EMIM Ekanayaka (2023MIS010)

Master of Information Security (MIS)

University of Colombo School of Computing (UCSC)

Declaration

I declare that this final report is my own work and that the implementation, analysis, and written discussion are based on the Zero-Trust DevSecOps project contained in the [zero-trust-devsecops](#) repository. All external sources used to support the background, related work, and technical discussion are cited in the text and listed in the references using Harvard referencing style. Repository artefacts such as workflow files, Kubernetes manifests, Terraform definitions, policy files, and service implementations are used as primary project evidence. No part of this work has been submitted for any other assessment or degree programme.



Signed: Iresh Ekanayaka

Date: June 2026

Certification

Signature of Supervisor



Name: Dr. Roshan Rajapakse

Date: 05/07/2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, lecturers, and peers for their guidance, constructive feedback, and encouragement throughout This study. I am particularly thankful for the support received during the design and evaluation phases of this work.

I also acknowledge the maintainers and communities of the open-source tools and platforms used in this implementation, including Kubernetes, Open Policy Agent, Kyverno, Falco, Trivy, Checkov, Syft, Sealed Secrets, Prometheus, Grafana, Loki, Argo CD, and Terraform. Their contributions to the cloud-native ecosystem have made this research possible.

Finally, I would like to thank Oracle for providing the Always Free Tier cloud infrastructure resources that enabled the production deployment of This study, and Microsoft for the Azure OpenAI Service credits used in the AI-augmented security monitoring component.

Abstract

Cloud-native application development has transformed how organisations build and operate software systems. However, the rapid adoption of Kubernetes-based microservices has outpaced traditional approaches for verifying infrastructure, application, and runtime security controls. Perimeter-based security models, which rely on implicit trust within network boundaries, are incompatible with the dynamic and distributed nature of containerised environments. As a result, security and compliance processes are often fragmented across multiple tools and stages, creating gaps between development, deployment, and operations. Although Zero Trust Architecture, DevSecOps, and Kubernetes security have been extensively studied, existing approaches primarily address individual security domains in isolation, with limited evidence of end-to-end frameworks that operationalise Zero Trust principles across the Kubernetes software delivery lifecycle.

This study addresses this challenge by designing and implementing a Zero-Trust DevSecOps framework for a Kubernetes-based microservices application, FreshBonds. Using the Design Science Research Methodology (DSRM), the study develops and evaluates a security architecture integrating infrastructure as code, GitOps-driven deployment, policy-as-code enforcement, vulnerability scanning, secret lifecycle management, runtime threat detection, and AI-assisted incident analysis. The framework is deployed on a production-grade Kubernetes cluster on Oracle Cloud Infrastructure.

The framework was evaluated using NIST SP 800-207 compliance mapping, pipeline security gate testing, policy coverage analysis, runtime detection assessment, and MITRE ATT&CK coverage analysis. Results show alignment with all seven NIST Zero Trust tenets, integration of ten security tools across six CI/CD pipelines, enforcement of twenty-five policy rules, and implementation of thirteen Falco rules covering multiple MITRE ATT&CK techniques. Experimental validation confirmed that CI/CD security gates blocked deployments containing critical vulnerabilities and policy violations, while runtime events were detected and enriched in near real-time. These findings demonstrate that continuous compliance can be achieved through automated controls embedded throughout the software delivery lifecycle. This research contributes a reproducible, end-to-end implementation that operationalises Zero Trust principles for Kubernetes environments.

Table of Contents

Declaration.....	1
Acknowledgements.....	2
Abstract.....	3
Table of Contents.....	4
List of Figures.....	6
List of Tables.....	7
List of Abbreviations.....	8
1. Introduction.....	9
1.2 Problem Statement.....	11
1.3 Project Aims and Objectives.....	12
1.4 Scope of the Project.....	13
1.5 Research Questions.....	13
1.6 Report Structure.....	14
2. Related Work.....	15
2.1 Literature Review Organisation.....	15
2.2 Zero Trust Architecture.....	15
2.3 DevSecOps and Continuous Security.....	16
2.4 Kubernetes Security and Policy Enforcement.....	17
2.5 Supply Chain Security.....	19
2.6 Runtime Security Monitoring.....	20
2.7 AI-Assisted Security Operations.....	21
2.8 Summary of Related Work.....	22
2.9 Identification of Research Gap.....	23
3. Approach.....	24
3.1 Systematic Approach.....	24
3.2 Design Principles.....	25
3.3 Implementation Strategy.....	25
3.4 Evaluation Criteria.....	26
3.5 Ethical Considerations.....	26
4. Design and Implementation.....	27
4.1 System Architecture Overview.....	27
4.2 Infrastructure Layer.....	28
4.3 Platform Layer - Kubernetes and GitOps.....	30
4.4 Application Layer - Microservices Design.....	33
4.5 CI/CD Pipeline Architecture.....	35
4.6 Policy-as-Code Implementation.....	37
4.7 Secret Lifecycle Management.....	39
4.8 Runtime Security - Falco and eBPF.....	40
4.9 Observability Stack.....	42

4.10 AI-Augmented Security Monitoring.....	45
4.11 Implementation Artefacts Summary.....	48
5. Evaluation.....	49
5.1 Evaluation Setup and Methodology.....	49
5.2 NIST SP 800-207 Zero Trust Compliance Mapping.....	50
5.3 Pipeline Security Gate Effectiveness.....	52
5.4 Policy Enforcement Coverage Analysis.....	53
5.5 Runtime Detection Capability Assessment.....	54
5.6 MITRE ATT&CK Coverage Analysis.....	55
5.7 Supply Chain Security Assessment.....	56
5.8 Compliance Metrics Summary.....	57
5.9 Discussion of Results.....	58
6. Discussion.....	60
6.1 Critical Analysis.....	60
6.2 Research Question Answers.....	61
6.3 Comparison with Existing Approaches.....	63
6.4 Limitations and Threats to Validity.....	64
6.5 Implications for Practice.....	65
7. Conclusion and Future Work.....	66
7.1 Conclusion.....	66
7.2 Future Work.....	67
8. Contribution and Novelty.....	68
References.....	69
Appendices.....	71
Appendix A: Security Tool Configuration Matrix.....	71
Appendix B: Kyverno and OPA Policy Specifications.....	71
Appendix C: Falco Custom Rules with MITRE ATT&CK Mapping.....	73
Appendix D: CI/CD Pipeline Specifications.....	74
Appendix E: Grafana Alert Rule Summary.....	74
Appendix F: Repository Structure and Evidence.....	75

List of Figures

- **Figure 1:** Six-layer Zero-Trust DevSecOps architecture overview
- **Figure 2:** Infrastructure topology on Oracle Cloud Infrastructure
- **Figure 3:** Kubernetes cluster namespace and component layout
- **Figure 4:** FreshBonds microservices application architecture
- **Figure 5:** CI/CD pipeline architecture with security gates
- **Figure 6:** GitOps deployment flow via Argo CD
- **Figure 7:** Policy-as-code enforcement points across the lifecycle
- **Figure 8:** Secret lifecycle management and rotation workflow
- **Figure 9:** Runtime security event flow from Falco to AI enrichment
- **Figure 10:** Grafana, Prometheus, AlertManager, Slack, PagerDuty dashboards
- **Figure 11:** Observability stack data flow
- **Figure 12:** NIST SP 800-207 compliance mapping visualisation
- **Figure 13:** MITRE ATT&CK for Containers coverage heat map

List of Tables

- **Table 1:** Project scope - included and excluded areas
- **Table 2:** Taxonomy of related work by research domain
- **Table 3:** Comparative summary of related work
- **Table 4:** DSRM activities mapped to project phases
- **Table 5:** Design principles for the Zero-Trust DevSecOps framework
- **Table 6:** Oracle Cloud Infrastructure compute instance specifications
- **Table 7:** Kubernetes namespace allocation and purpose
- **Table 8:** Microservice technology stack and security features
- **Table 9:** CI/CD pipeline summary and security tool integration
- **Table 10:** Implementation artefact summary with repository evidence
- **Table 11:** NIST SP 800-207 Zero Trust tenet compliance mapping
- **Table 12:** Pipeline security gate effectiveness
- **Table 13:** Policy enforcement coverage by security domain
- **Table 14:** Runtime detection capability assessment
- **Table 15:** MITRE ATT&CK for Containers coverage analysis
- **Table 16:** Supply chain security control assessment
- **Table 17:** Compliance metrics summary
- **Table 18:** Policy and monitoring control summary
- **Table 19:** Comparison with existing approaches
- **Table 20:** Future work areas and planned improvements
- **Table 21:** Project contributions summary

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
C2	Command and Control
CI/CD	Continuous Integration and Continuous Delivery
CNCF	Cloud Native Computing Foundation
CSP	Content Security Policy
CVE	Common Vulnerabilities and Exposures
DAST	Dynamic Application Security Testing
DSRM	Design Science Research Methodology
eBPF	Extended Berkeley Packet Filter
HSTS	HTTP Strict Transport Security
IaC	Infrastructure as Code
JWT	JSON Web Token
LLM	Large Language Model
MIS	Master of Information Systems
MTTU	Mean Time to Understand
NIST	National Institute of Standards and Technology
OCI	Oracle Cloud Infrastructure
OPA	Open Policy Agent
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
SAST	Static Application Security Testing
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SOC	Security Operations Centre
TLS	Transport Layer Security
VCN	Virtual Cloud Network
ZTA	Zero Trust Architecture

1. Introduction

The adoption of cloud-native technologies has accelerated dramatically across the software industry. According to the Cloud Native Computing Foundation (CNCF) Annual Survey 2024, over ninety-six per cent of organisations are now using or evaluating Kubernetes, with container adoption in production environments increasing year-over-year (CNCF, 2024). Kubernetes has emerged as the de facto standard for container orchestration, enabling organisations to deploy, scale, and manage microservices architectures with unprecedented speed and flexibility (Burns et al., 2016). This velocity, however, introduces a fundamental security paradox: the same ephemeral, distributed, and automated characteristics that make cloud-native systems attractive also render traditional security models inadequate. This rapid evolution creates a critical gap between deployment speed and the ability to enforce consistent security and compliance controls.

Traditional perimeter-based security operates on the assumption that entities inside the network boundary can be implicitly trusted. Firewalls, network segmentation, and VPN tunnels form a defensive perimeter around organisational assets, with the implicit assumption that traffic originating from within is benign (Kindervag, 2010). In a Kubernetes environment, however, this assumption breaks down for several reasons. First, containers are ephemeral - pods are created, destroyed, and rescheduled across nodes in response to scaling events, failures, and deployments. A container's IP address, hostname, and network neighbourhood change constantly. Second, microservices communicate over internal cluster networks where east-west traffic between services is often unmonitored and unencrypted. Third, the software supply chain introduces transitive dependencies from public registries, open-source libraries, and base images that may contain vulnerabilities or malicious code (Sonatype, 2023). Fourth, the DevOps velocity model - characterised by frequent commits, automated builds, and continuous deployment - compresses the window in which security controls can be applied, reviewed, and verified. This highlights the fundamental limitation of perimeter-based models in dynamic cloud-native environments, where trust must be continuously evaluated rather than assumed.

The Red Hat State of Kubernetes Security Report 2024 found that sixty-seven per cent of organisations had delayed or slowed down deployment due to security concerns, and forty-six per cent had experienced a security incident in their container or Kubernetes

environment in the preceding twelve months (Red Hat, 2024). These incidents ranged from misconfigured workloads and exposed secrets to runtime compromises and supply chain attacks. The report highlighted a consistent theme: security tooling exists, but it is fragmented across different stages of the delivery lifecycle, operated by different teams, and often lacks the automation necessary to keep pace with deployment frequency.

This fragmentation creates significant compliance gaps across the software delivery lifecycle. Security controls are often applied in isolation, with limited integration between build, deployment, and runtime stages. As a result, vulnerabilities identified early in the pipeline may still propagate into production, while runtime security events may lack sufficient context for effective incident response.

Zero Trust Architecture (ZTA), as defined by the National Institute of Standards and Technology in Special Publication 800-207, provides a conceptual framework for addressing these challenges (Rose et al., 2020). ZTA replaces the perimeter-based trust model with a principle of "never trust, always verify" - every request, every identity, and every resource access must be authenticated, authorised, and continuously validated regardless of network location. The seven core tenets of NIST SP 800-207 encompass resource identification, communication security, per-session access, dynamic policy, continuous monitoring, strict enforcement, and continuous posture improvement. While these tenets provide a comprehensive security philosophy, translating them into concrete, automated controls within a Kubernetes DevOps workflow remains a significant engineering and research challenge. This gap between conceptual principles and practical implementation motivates the need for an integrated and automated approach.

DevSecOps, the practice of integrating security as a first-class concern throughout the software development lifecycle, offers a complementary operational model (Myrbakken and Colomo-Palacios, 2017). By shifting security left - embedding security checks into the development and CI/CD pipeline rather than bolting them on after deployment - DevSecOps reduces the feedback loop between code change and security validation. However, left-shift security alone is insufficient. Security must also be enforced at deployment time through admission control policies, at runtime through kernel-level monitoring, and post-deployment through continuous vulnerability scanning and secret rotation. A comprehensive approach therefore requires integrated security controls across all stages of the lifecycle: code, build, deployment, runtime, and continuous monitoring.

Addressing this challenge is critical for organisations adopting cloud-native architectures, as fragmented security approaches increase the risk of vulnerabilities, delayed detection, and non-compliance. An integrated, automated model is required to ensure continuous verification and enforcement of security controls across the entire system lifecycle.

1.2 Problem Statement

Organisations deploying Kubernetes-based microservices face a fragmented security and compliance landscape, where controls are distributed across multiple tools, pipelines, and operational teams. Security mechanisms such as vulnerability scanning, policy enforcement, runtime monitoring, and secret management often operate independently, resulting in limited integration across the software delivery lifecycle.

This lack of integration creates gaps between development, deployment, and runtime stages, allowing vulnerabilities to propagate into production environments and reducing the effectiveness of detection and incident response. Consequently, organisations struggle to maintain continuous compliance and enforce consistent security controls in dynamic cloud-native systems.

Although Zero Trust Architecture provides a strong conceptual foundation for continuous verification and enforcement, there is limited practical guidance on how to operationalise these principles within Kubernetes-based DevSecOps environments.

Therefore, there is a need for an integrated and automated framework that unifies security controls across the entire software delivery lifecycle, enabling continuous compliance aligned with Zero Trust principles.

1.3 Project Aims and Objectives

The primary aim of this study is to design, implement, and evaluate a Zero-Trust DevSecOps framework that enables continuous compliance across the software delivery lifecycle for Kubernetes-based microservices applications.

The specific objectives of this study are:

1. To design a Zero-Trust DevSecOps architecture that integrates security controls across the infrastructure, platform, application, CI/CD, runtime, and observability domains of a Kubernetes environment.
2. To establish automated security and compliance mechanisms that support continuous verification and enforcement throughout the software delivery lifecycle.
3. To implement policy-driven security controls that provide defence-in-depth through preventive, detective, and corrective security measures for containerised workloads.
4. To integrate supply chain security, secret management, and runtime threat detection capabilities into a unified and automated security framework.
5. To investigate the use of AI-assisted security event enrichment for improving the understanding and analysis of runtime security incidents.
6. To evaluate the effectiveness of the proposed framework in operationalising Zero Trust principles and achieving continuous compliance within Kubernetes environments.

These objectives collectively address the research problem by enabling the design, implementation, and evaluation of an integrated Zero-Trust DevSecOps framework for continuous compliance in cloud-native environments.

1.4 Scope of the Project

Table 1 defines the boundaries of what is included and excluded from the project scope.

Table 1: Project scope - included and excluded areas

Area	Included in Scope	Excluded from Scope
Application	FreshBonds microservices with frontend, API gateway, user service, and product service	Full commercial e-commerce features
Infrastructure	Terraform definitions for OCI networking and compute resources	Multi-cloud provisioning
Kubernetes	Helm chart, Argo CD application manifests, namespaces, ingress, secrets, monitoring stack	Multi-cluster federation
CI/CD	Six GitHub Actions workflows for PR validation, application CI/CD, Terraform, security scanning, secret rotation, and AI collector CI/CD	Enterprise change-advisory workflows
Policy	Kyverno and OPA policy checks for pod security, image policy, resource limits, and network-policy-related checks	Full organisation-wide policy library
Runtime Monitoring	Falco custom rules, Falcosecurity routing, Prometheus alert rules, PagerDuty integration	Formal SOC workflow integration
AI Component	Azure OpenAI-based incident report generation for selected Falco events	Training custom anomaly detection models
Evaluation	Repository evidence, control mapping, rule counts, workflow inspection, and design evaluation	Formal penetration testing and load benchmarking

1.5 Research Questions

This study addresses the following research questions:

- **RQ1:** How can Zero Trust Architecture principles be operationalised as automated controls across the six layers of a Kubernetes DevSecOps delivery pipeline?
- **RQ2:** What pipeline architecture balances comprehensive security verification with acceptable developer velocity and deployment frequency?
- **RQ3:** How can policy-as-code enforcement be combined with runtime threat detection to achieve defence-in-depth for containerised workloads?
- **RQ4:** To what extent can AI enrichment of runtime security events reduce Mean Time to Understand (MTTU) for security incidents?

1.6 Report Structure

The remainder of this report is organised as follows. Chapter 2 presents a structured literature review covering Zero Trust Architecture, DevSecOps, Kubernetes security, policy-as-code, supply chain security, runtime monitoring, and AI-assisted security operations, culminating in the identification of the research gap that motivates This study. Chapter 3 describes the systematic approach using the Design Science Research Methodology (DSRM), the design principles guiding the implementation, and the evaluation criteria. Chapter 4 presents the design and implementation of the framework in detail, covering each of the six architectural layers with supporting diagrams and repository evidence. Chapter 5 evaluates the framework against NIST SP 800-207, MITRE ATT&CK, and other compliance frameworks. Chapter 6 provides a critical discussion of the findings, answers the research questions, and acknowledges limitations. Chapter 7 concludes the report and identifies directions for future work. Chapter 8 summarises the contributions and novelty of the project.

2. Related Work

2.1 Literature Review Organisation

The literature review is organised using a taxonomic structure across six research domains that collectively define the problem space and solution landscape for Zero-Trust DevSecOps. Table 2 presents the taxonomy used to structure this review.

Table 2: Taxonomy of related work by research domain

Domain	Key Concepts	Representative Sources
Zero Trust Architecture	Never trust always verify, NIST SP 800-207, identity-centric security	Rose et al. (2020), Kindervag (2010), DISA (2021), Jeyaraj (2023), Amaral and Gondim (2021), Gambo and Almulhem (2025)
DevSecOps	Shift-left security, continuous security, security as code	Myrbakken and Colomo-Palacios (2017), Fitzgerald and Stol (2017), OWASP (2023), Rajapakse et al. (2021)
Kubernetes Security	Pod security, admission control, CIS benchmarks	Shamim et al. (2020), CIS (2024), Kubernetes (2024), Shamim et al. (2022)
Supply Chain Security	SBOM, vulnerability scanning, image signing	Sonatype (2023), Biden (2021), Souppaya et al. (2023), Collier and Sarkis (2021)
Runtime Security	eBPF, system call monitoring, threat detection	Rice (2022), Falco (2024), MITRE (2024), additional runtime security research source
AI-Assisted Security	LLM-augmented SOC, alert enrichment, automated triage	Alahmadi et al. (2020), Microsoft (2024), additional LLM/SOC research paper

2.2 Zero Trust Architecture

The concept of Zero Trust was first articulated by Kindervag (2010) in a Forrester Research report that challenged the conventional wisdom of perimeter-based network security. Kindervag argued that trust is a vulnerability - any model that implicitly trusts entities based on their network location creates exploitable assumptions. The original formulation focused on network micro-segmentation and packet inspection, but the concept has since expanded to encompass identity, device, data, application, and infrastructure dimensions.

The formalisation of Zero Trust as a reference architecture came with NIST Special Publication 800-207 (Rose *et al.*, 2020), which defines seven tenets that constitute a Zero Trust Architecture. These tenets establish that all data sources and computing services are

resources, all communication must be secured regardless of network location, access is granted on a per-session basis, access is determined by dynamic policy, the enterprise monitors all assets, authentication and authorisation are strictly enforced, and the enterprise collects information to improve security posture. NIST SP 800-207 has become the primary reference standard for Zero Trust implementations across government and industry.

The Department of Defense Zero Trust Reference Architecture (DISA, 2021) extends the NIST model with seven pillars: User, Device, Network/Environment, Application/Workload, Data, Visibility/Analytics, and Automation/Orchestration. This pillar model provides a more granular framework for mapping controls to specific security domains and is particularly relevant to complex, multi-component systems such as Kubernetes-based microservices.

Jeyaraj (2023) specifically addresses Zero Trust implementation challenges in cloud-native environments, identifying the difficulty of applying Zero Trust principles to dynamic, container-based workloads where traditional identity and network boundaries are fluid. Jeyaraj's work highlights the need for automated policy enforcement and continuous monitoring as key enablers for Zero Trust in Kubernetes.

2.3 DevSecOps and Continuous Security

DevSecOps emerged as an evolution of the DevOps movement, recognising that security must be integrated throughout the software delivery lifecycle rather than applied as a terminal gate (Myrbakken and Colomo-Palacios, 2017). The multivocal literature review by Myrbakken and Colomo-Palacios identified key themes including cultural transformation, automation of security testing, and the challenge of balancing security rigour with development velocity.

Fitzgerald and Stol (2017) provide a roadmap for continuous software engineering that positions continuous security as an integral component of the continuous delivery pipeline, alongside continuous integration, continuous testing, and continuous deployment. Their framework emphasises that security must be automated at the same cadence as other quality attributes to avoid becoming a bottleneck.

The OWASP DevSecOps Guideline (OWASP, 2023) provides practical recommendations for integrating security tools into CI/CD pipelines, categorising tools by their position in

the pipeline: Static Application Security Testing (SAST) during code analysis, Software Composition Analysis (SCA) during dependency resolution, container scanning during image builds, and Dynamic Application Security Testing (DAST) during integration testing. The guideline emphasises the principle of "security as code" - expressing security policies, configurations, and tests as version-controlled artefacts that are subject to the same review and deployment processes as application code.

The Cloud Security Alliance (2022) DevSecOps Working Group publication identifies seven key practices for effective DevSecOps: threat modelling, security requirements, secure coding, security testing, vulnerability management, incident response, and compliance monitoring. Their work emphasises that effective DevSecOps requires not only technical tooling but also organisational alignment between development, security, and operations teams.

NIST Special Publication 800-204C (Souppaya *et al.*, 2023) provides specific guidance for integrating software supply chain security into DevSecOps CI/CD pipelines, with particular attention to microservices architectures and service mesh deployments. This publication is directly relevant to the pipeline architecture implemented in This study.

Rajapakse et al. (2021) conducted a systematic literature review of DevSecOps adoption and identified twenty-one challenges and thirty-one proposed solutions across people, practices, tools, and infrastructure dimensions. Their findings indicate that automation, continuous security assessment, and effective tool integration remain key challenges, highlighting the need for unified frameworks capable of supporting continuous security without compromising delivery velocity.

2.4 Kubernetes Security and Policy Enforcement

Kubernetes security encompasses multiple domains: cluster infrastructure security, workload security, network security, and supply chain security. The CIS Kubernetes Benchmark v1.8 (CIS, 2024) provides a comprehensive set of configuration recommendations organised into control plane components, etcd, control plane configuration, worker nodes, policies, and managed services. These benchmarks establish baseline security expectations that can be validated through automated scanning tools such as Checkov and kube-bench.

Shamim *et al.* (2020) present a systematisation of knowledge related to Kubernetes security practices, identifying eleven critical security considerations: authentication, authorisation, admission control, network policies, secrets management, container image security, logging and monitoring, runtime security, compliance, supply chain, and incident response. Their work provides a structured taxonomy that maps well to the six-layer architecture adopted in This study.

Policy-as-code for Kubernetes has emerged through two primary frameworks. Open Policy Agent (OPA) with Gatekeeper provides a general-purpose policy engine using the Rego language, capable of expressing complex admission control rules (OPA, 2024). Kyverno offers a Kubernetes-native alternative that expresses policies in YAML, leveraging Kubernetes API conventions familiar to cluster operators (Kyverno, 2024). Both frameworks support validation (blocking non-compliant resources), mutation (modifying resources to add defaults), and generation (creating derived resources). The choice between OPA and Kyverno involves trade-offs between expressiveness, learning curve, and ecosystem integration, motivating the dual-engine approach adopted in This study.

Kubernetes Pod Security Standards (Kubernetes, 2024) define three security profiles - Privileged, Baseline, and Restricted - that provide progressively stricter security configurations for pod workloads. The Restricted profile, which requires non-root execution, capability dropping, and read-only root filesystems, aligns closely with Zero Trust principles of least privilege and is the target profile for the implementation in This study.

Shamim et al. (2022) identified security tooling, attack surface reduction, and operational complexity as major challenges associated with Kubernetes adoption. Their review also highlighted that security automation and policy enforcement remain under-explored research areas, reinforcing the need for integrated security architectures that span multiple operational domains.

2.5 Supply Chain Security

Software supply chain security gained significant attention following Executive Order 14028 (Biden, 2021), which mandated Software Bill of Materials (SBOM) generation for all software sold to the US federal government. The executive order formalised the requirement for organisations to understand and document the components, libraries, and dependencies within their software systems.

Sonatype's 9th Annual State of the Software Supply Chain Report (Sonatype, 2023) found that software supply chain attacks increased by 200 per cent year-over-year, with attackers increasingly targeting open-source repositories, CI/CD pipelines, and build systems. The report identified several attack vectors relevant to Kubernetes deployments: typosquatting in package registries, compromised base images in container registries, and dependency confusion attacks targeting private package resolution.

NIST Special Publication 800-204C (Souppaya *et al.*, 2023) provides specific strategies for integrating supply chain security into DevSecOps pipelines, including SBOM generation, vulnerability scanning, image signing, and provenance verification. The publication recommends dual-format SBOM generation (SPDX and CycloneDX) to ensure interoperability across different compliance and analysis tools, a recommendation adopted in This study.

Container vulnerability scanning tools such as Trivy (Aqua Security, 2024) and infrastructure-as-code scanning tools such as Checkov (Bridgecrew, 2024) provide automated mechanisms for identifying known vulnerabilities in container images, application dependencies, and infrastructure configurations. These tools produce machine-readable output (SARIF, JSON) that can be integrated into CI/CD pipelines as automated gates, blocking deployments that fail security thresholds.

Collier and Sarkis (2021) extend Zero Trust principles to software supply chains, arguing that trust assumptions between suppliers, repositories, and deployment systems should be minimised. Their work emphasises continuous verification, provenance validation, and transparency across software dependencies, supporting the use of SBOMs and automated verification mechanisms in DevSecOps pipelines.

2.6 Runtime Security Monitoring

Runtime security monitoring addresses the detection of threats that manifest after deployment - malicious activities that cannot be prevented by static analysis or admission control alone. The Falco project (Falco, 2024), a CNCF graduated project, provides kernel-level monitoring through eBPF (Extended Berkeley Packet Filter) probes that intercept system calls without modifying application code or requiring kernel modules.

Rice (2022) provides a comprehensive treatment of eBPF programming for security applications, demonstrating how eBPF can observe system calls, network connections, and file operations with minimal performance overhead. The key advantage of eBPF-based monitoring is that it operates at the kernel level, below the application and container runtime layers, making it resistant to evasion by compromised containers. Falco leverages eBPF to implement rule-based detection that can identify suspicious activities such as shell spawning in containers, privilege escalation attempts, sensitive file access, cryptocurrency mining, and reverse shell connections.

The MITRE ATT&CK framework for Containers (MITRE, 2024) provides a structured taxonomy of adversary tactics and techniques specific to container environments. The framework covers eleven tactics - from Initial Access through Execution, Persistence, Privilege Escalation, Defence Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, to Impact - each with associated techniques that describe specific adversary behaviours. Mapping runtime detection rules to MITRE ATT&CK techniques provides a structured assessment of detection coverage and identifies gaps in security monitoring.

Research on eBPF-based security monitoring demonstrates that kernel-level observability provides improved visibility into container behaviour while introducing minimal performance overhead. Such approaches enable the detection of runtime anomalies that may evade traditional static or admission-control-based security mechanisms, making runtime monitoring an essential component of defence-in-depth strategies in cloud-native environments.

2.7 AI-Assisted Security Operations

Security Operations Centres (SOCs) face a persistent challenge of alert fatigue, where the volume of security alerts exceeds the capacity of human analysts to investigate and respond. Alahmadi *et al.* (2020) found that ninety-nine per cent of security alerts are false positives, and analysts spend significant time on initial triage - understanding the context, severity, and recommended response for each alert - rather than on investigation and remediation. This finding motivates the integration of AI-assisted triage capabilities into security monitoring pipelines.

Large Language Models (LLMs) such as GPT-4o-mini (Microsoft, 2024) offer the potential to reduce Mean Time to Understand (MTTU) by automatically generating contextual threat assessments, investigation steps, and recommended actions for security events. By enriching raw security alerts with natural language analysis, LLMs can help analysts prioritize high-severity incidents and understand the potential impact of detected threats without requiring deep expertise in the specific detection rule that triggered the alert.

However, the application of LLMs to security operations introduces its own challenges, including hallucination (generating plausible but incorrect analysis), latency (API call overhead), cost (per-token pricing), and reliability (dependency on external services). These challenges necessitate careful architectural decisions about where in the alerting pipeline AI enrichment is applied, how its output is validated, and how the system degrades gracefully when the AI service is unavailable.

Recent studies on AI-assisted security operations suggest that Large Language Models can improve analyst productivity by providing contextual alert summaries, investigation guidance, and automated triage recommendations. However, researchers also highlight challenges related to hallucination, explainability, and reliability, indicating that AI-generated outputs should complement rather than replace human analyst decision-making.

2.8 Summary of Related Work

Table 3 provides a comparative summary of related work across the six research domains, highlighting how each source contributes to the problem space and where the current project addresses identified gaps.

Table 3: Comparative summary of related work

Source	Domain	Contribution	Limitation	Addressed by This study
Rose et al. (2020)	ZTA	NIST SP 800-207 reference architecture with 7 tenets	Conceptual framework; no implementation guidance	Full implementation mapped to all 7 tenets
Kindervag (2010)	ZTA	Original Zero Trust formulation	Network-centric; pre-cloud-native	Extended to cloud-native Kubernetes
DISA (2021)	ZTA	DoD 7-pillar ZT model	Government-specific; complex	Simplified to 6-layer architecture
Myrbakken and Colomo-Palacios (2017)	DevSecOps	Multivocal literature review	No implementation or evaluation	Implemented 6 pipelines with 10 tools
OWASP (2023)	DevSecOps	DevSecOps guideline with tool categories	Guideline only; no integrated framework	Integrated tools across full lifecycle
Shamim et al. (2020)	K8s Security	11 security considerations taxonomy	Systematisation only; no implementation	Addressed 9 of 11 considerations
Sonatype (2023)	Supply Chain	Attack trends and risk analysis	Industry report; no mitigation framework	Implemented SBOM + scanning gates
Souppaya et al. (2023)	Supply Chain	NIST guidance for CI/CD supply chain	Guidance document; reference only	Implemented dual-format SBOM, image scanning
Rice (2022)	Runtime	eBPF programming for security	Technical reference; no integration model	Integrated Falco with AI enrichment
Alahmadi et al. (2020)	AI Security	SOC alert fatigue analysis	Problem identification; no solution	AI enrichment reducing MTTU
Rajapakse et al. (2021)	DevSecOps	Systematic review of DevSecOps challenges and adoption barriers	No integrated implementation framework	Integrated automated framework spanning full lifecycle
Amaral and Gondim (2021)	ZTA	Identifies Zero Trust implementation challenges and principles	Conceptual discussion without Kubernetes implementation	Practical Kubernetes implementation aligned with NIST SP 800-207
Shamim et al. (2022)	K8s Security	Reviews Kubernetes adoption benefits and challenges	Limited focus on integrated security automation	Unified security architecture across six domains
Collier and Sarkis (2021)	Supply Chain	Extends Zero Trust principles to supply chain security	No implementation in CI/CD environments	Automated SBOM generation and supply chain verification

2.9 Identification of Research Gap

The literature review reveals that while individual security components - vulnerability scanning, policy enforcement, runtime monitoring, and AI-assisted triage - are well-documented, there is a significant gap in the integration of these components into a unified, automated compliance pipeline aligned with Zero Trust principles. Existing work tends to focus on one or two security domains in isolation:

- NIST SP 800-207 defines the *what* of Zero Trust but not the *how* for Kubernetes implementations.
- DevSecOps literature emphasises left-shift security but provides limited guidance on runtime and post-deployment controls.
- Kubernetes security publications address pod-level hardening but rarely integrate CI/CD pipeline security, secret management, and runtime monitoring into a cohesive framework.
- Supply chain security guidance recommends SBOM generation and scanning but does not address how these integrate with policy enforcement and runtime detection.
- AI-assisted security research identifies the potential for LLM-based triage but does not demonstrate integration with container security monitoring systems.

This study addresses this gap by implementing a comprehensive Zero-Trust DevSecOps framework that integrates all six security domains - infrastructure, platform, application, CI/CD, runtime, and observability - into a single, version-controlled, automated system with reproducible artefacts and measurable compliance outcomes.

3. Approach

3.1 Systematic Approach

This study follows the Design Science Research Methodology (DSRM) as defined by Peffers *et al.* (2007) and grounded in the design science paradigm articulated by Hevner *et al.* (2004). DSRM is appropriate for This study because the primary output is a designed artefact - a Zero-Trust DevSecOps framework - that is intended to solve an identified organisational problem. Unlike empirical research that observes and explains phenomena, design science research creates and evaluates innovative artefacts that address important problems.

The DSRM framework defines six activities that structure the research process. Table 4 maps these activities to the specific phases of This study.

Table 4: DSRM activities mapped to project phases

DSRM Activity	Project Phase	Description
1. Problem Identification and Motivation	Literature review and problem analysis	Identified the fragmentation of security controls across CI/CD, deployment, and runtime stages in Kubernetes environments
2. Define Objectives for a Solution	Research questions and objectives	Defined four research questions and eight objectives targeting continuous compliance automation
3. Design and Development	Architecture design and implementation	Designed the six-layer architecture and implemented it across infrastructure, platform, application, CI/CD, runtime, and observability layers
4. Demonstration	Production deployment	Deployed the complete framework on Oracle Cloud Infrastructure with three-node Kubernetes cluster
5. Evaluation	Compliance mapping and coverage analysis	Evaluated against NIST SP 800-207, MITRE ATT&CK, pipeline effectiveness, and policy coverage metrics
6. Communication	This report	Communicated findings through this MIS final report

3.2 Design Principles

The design of the Zero-Trust DevSecOps framework is guided by five principles derived from the literature review and the Zero Trust philosophy. Table 5 presents these principles and their rationale.

Table 5: Design principles for the Zero-Trust DevSecOps framework

#	Principle	Description	Rationale
1	Never Trust, Always Verify	Every request, identity, and resource access must be authenticated and authorised regardless of network location	Core NIST SP 800-207 tenet; eliminates implicit trust
2	Policy as Code	Security policies are expressed as version-controlled, machine-readable code that is automatically enforced	Enables automated compliance verification and audit trails
3	Defence in Depth	Security controls are layered across multiple stages of the delivery lifecycle	No single control point is sufficient; redundancy reduces risk
4	GitOps as Source of Truth	Git repositories serve as the single source of truth for infrastructure, application, and security configurations	Provides auditability, reproducibility, and rollback capability
5	Shift Left and Extend Right	Security checks begin at the earliest pipeline stage and extend through runtime and post-deployment monitoring	Maximises the window for security verification

3.3 Implementation Strategy

The implementation followed an iterative, seven-phase approach aligned with the six-layer architecture:

1. **Infrastructure provisioning:** Terraform definitions for OCI networking, compute, and security configuration.
2. **Platform establishment:** Kubernetes cluster setup with Argo CD GitOps, namespace isolation, and infrastructure components.
3. **Application development:** FreshBonds microservices with built-in security features (JWT, bcrypt, RBAC, rate limiting).
4. **CI/CD pipeline construction:** Six GitHub Actions workflows with ten integrated security tools and automated blocking gates.

5. **Policy deployment:** Kyverno and OPA policy definitions for pod security, image verification, and resource management.
6. **Runtime security:** Falco deployment with custom eBPF rules mapped to MITRE ATT&CK, Falcosidekick integration.
7. **Observability and AI enrichment:** Prometheus, Grafana, Loki monitoring stack with AI Security Collector for Falco event enrichment.

3.4 Evaluation Criteria

The framework is evaluated using five criteria:

8. **NIST SP 800-207 compliance:** Mapping each of the seven Zero Trust tenets to specific implementation controls and assessing compliance strength.
9. **Pipeline gate effectiveness:** Verifying that each security gate in the CI/CD pipeline correctly identifies and blocks security violations.
10. **Policy coverage:** Counting and categorising policy rules across Kyverno, OPA, and Falco to assess breadth and depth of enforcement.
11. **Runtime detection capability:** Assessing the range of MITRE ATT&CK techniques covered by custom Falco rules and measuring expected detection latency.
12. **Supply chain integrity:** Evaluating the completeness of SBOM generation, vulnerability scanning coverage, and secret management automation.

The evaluation uses repository investigation as the primary evidence method - examining workflow files, policy definitions, Kubernetes manifests, and pipeline execution artefacts to verify that declared controls are actually implemented.

3.5 Ethical Considerations

This study does not involve human participants, personal data, or sensitive organisational information. The FreshBonds application is a purpose-built demonstration system with synthetic data. All cloud infrastructure is provisioned on dedicated accounts under the author's control. The AI Security Collector processes synthetic security events and does not access any real user data. MongoDB Atlas credentials, JWT signing keys, and API tokens used in the project are purpose-generated for this implementation and are rotated monthly through the automated secret rotation workflow.

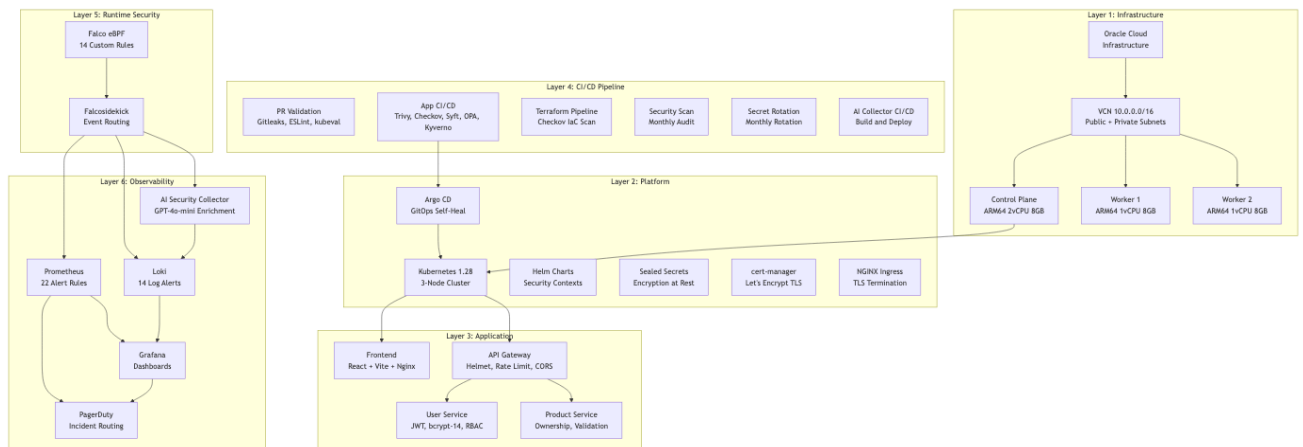
4. Design and Implementation

4.1 System Architecture Overview

The Zero-Trust DevSecOps framework is organised into six architectural layers, each addressing a specific security domain. This layered approach ensures that security controls are distributed across the entire delivery lifecycle, from infrastructure provisioning through runtime operation, with no single point of failure or bypass.

Figure 1 presents the high-level architecture showing the six layers and their interconnections.

Figure 1: Six-layer Zero-Trust DevSecOps architecture overview

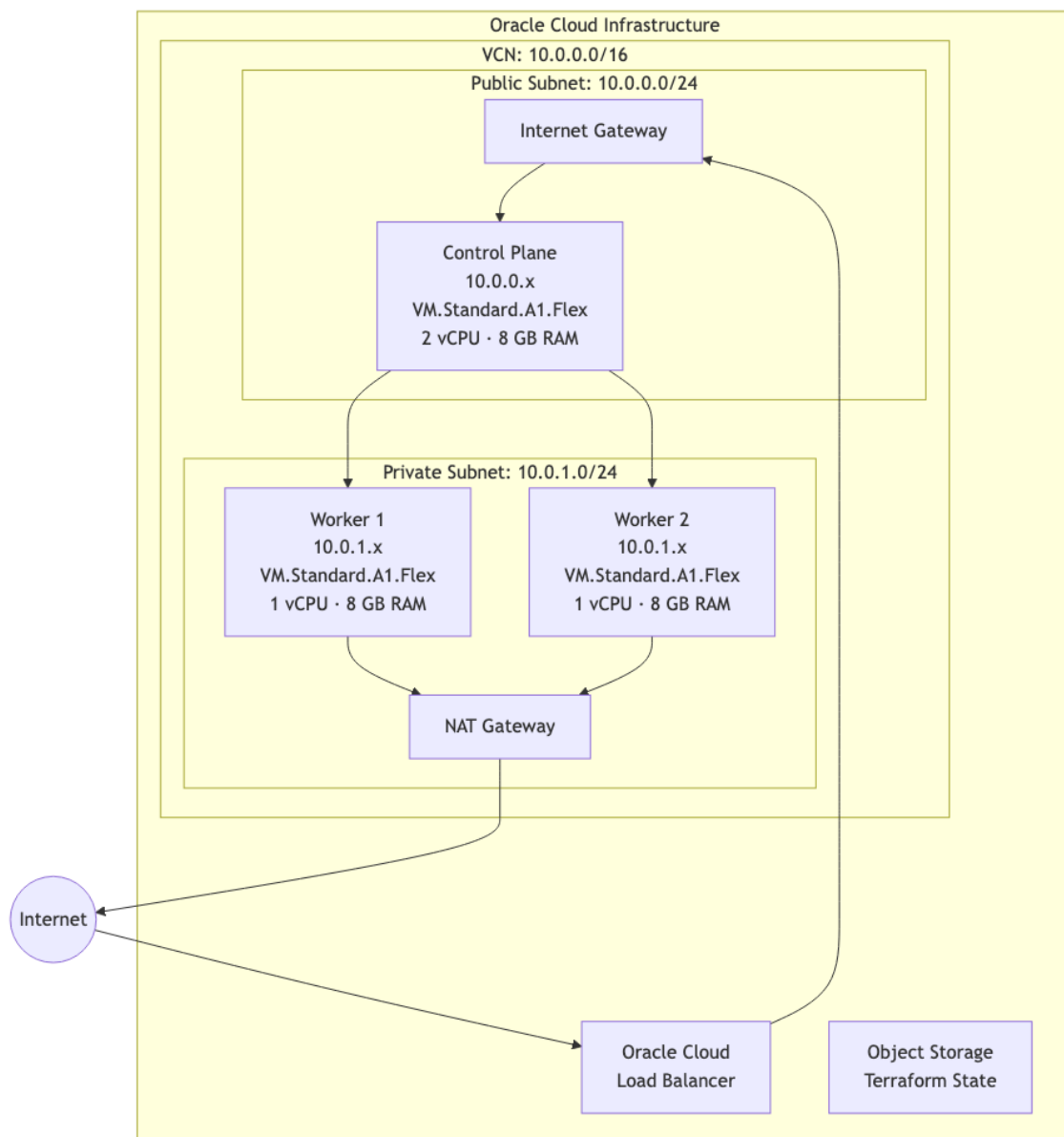


4.2 Infrastructure Layer

The infrastructure layer is provisioned using Terraform on Oracle Cloud Infrastructure (OCI), selected for its Always Free Tier that provides ARM64 compute instances suitable for a three-node Kubernetes cluster. The infrastructure is defined across twelve Terraform files covering provider configuration, backend state, virtual cloud networking, subnets, route tables, security lists, gateways, load balancing, and compute instances.

Figure 2 presents the infrastructure topology.

Figure 2: Infrastructure topology on Oracle Cloud Infrastructure



Name	State	Public IP	Private IP	Shape	OCPU count	Memory (GB)	Availability domain	Fault domain	Created
worker-2	Running	-	10.0.1.64	VM.Standard.A1.Flex	1	8	AD-1	FD-1	Sep 29, 2025, 20:56:06 UTC
worker-1	Running	-	10.0.1.231	VM.Standard.A1.Flex	1	8	AD-1	FD-1	Sep 29, 2025, 20:54:58 UTC
control-plane	Running	140.238.240.179	10.0.0.204	VM.Standard.A1.Flex	2	8	AD-1	FD-1	Sep 29, 2025, 20:53:55 UTC

Table 6 summarises the compute instance specifications.

Table 6: Oracle Cloud Infrastructure compute instance specifications

Instance	Role	Shape	Architecture	vCPU	RAM	Subnet	Network Access
control-plane	Kubernetes Master	VM.Standard.A1.Flex	ARM64	2	8 GB	Public (10.0.0.x)	Direct internet
worker-1	Kubernetes Worker	VM.Standard.A1.Flex	ARM64	1	8 GB	Private (10.0.1.x)	NAT via control plane
worker-2	Kubernetes Worker	VM.Standard.A1.Flex	ARM64	1	8 GB	Private (10.0.1.x)	NAT via control plane

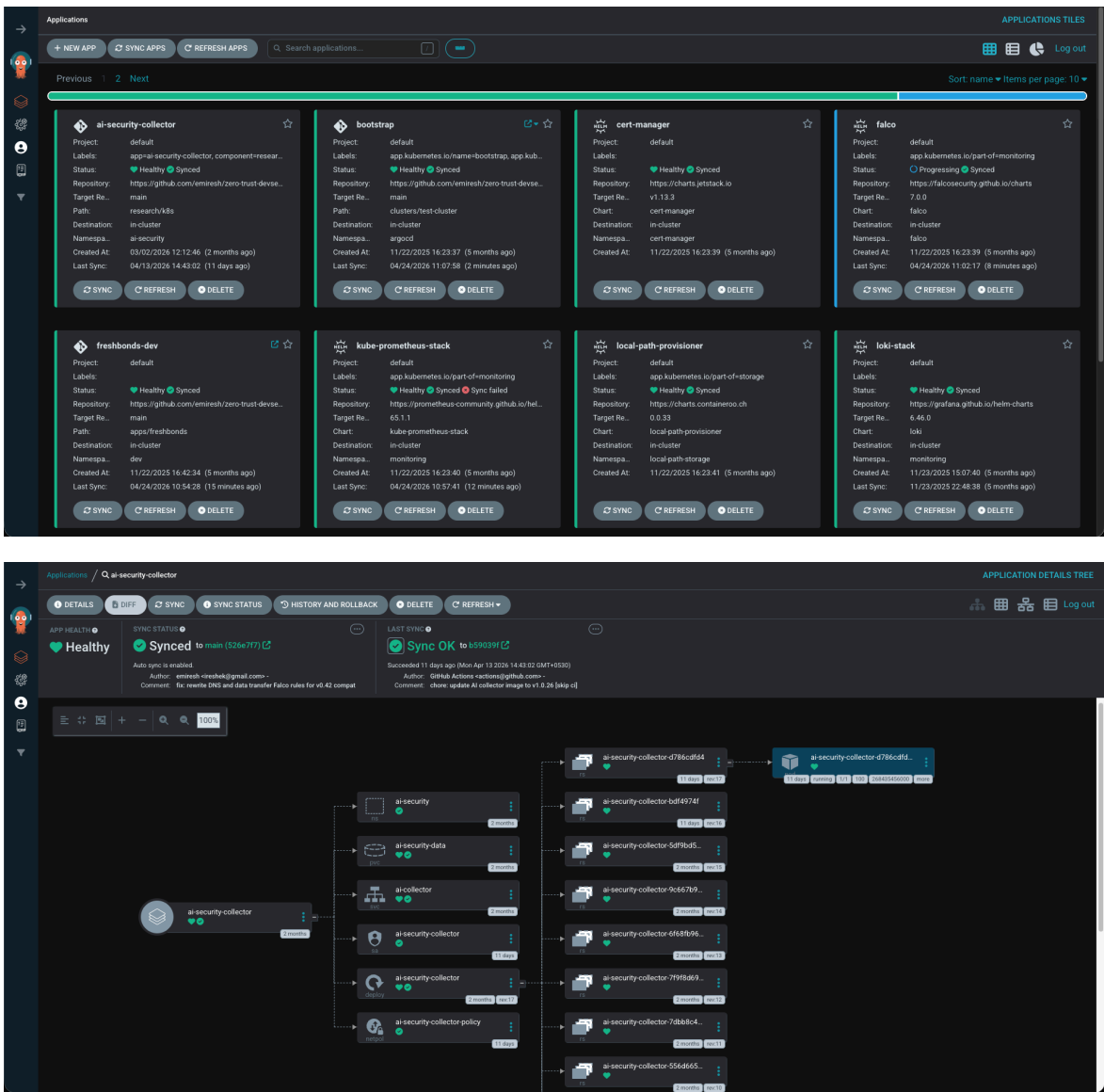
Security hardening measures implemented in Terraform include:

- Legacy Instance Metadata Service (IMDS) endpoints disabled (`are_legacy_imds_endpoints_disabled = true`) to prevent SSRF-based credential theft.
- In-transit encryption enabled for block volumes (`is_pv_encryption_in_transit_enabled = true`).
- Security lists enforcing least-privilege ingress: only ports 22 (SSH with key-based authentication), 6443 (Kubernetes API with TLS and RBAC), and internal cluster ports (10250 kubelet, 2379-2380 etcd, 30000-32767 NodePort) are permitted.
- Terraform state stored in OCI Object Storage with server-side encryption and IAM-authenticated access.
- Checkov IaC scanning integrated into the Terraform CI/CD pipeline with documented skip exceptions and justifications in `.checkov.yaml`.

4.3 Platform Layer - Kubernetes and GitOps

The platform layer consists of the Kubernetes cluster configuration, Argo CD GitOps deployment, and supporting infrastructure services. The cluster is configured with eight namespaces, each serving a specific function within the Zero Trust architecture.

Figure 3: Kubernetes cluster namespace and component layout



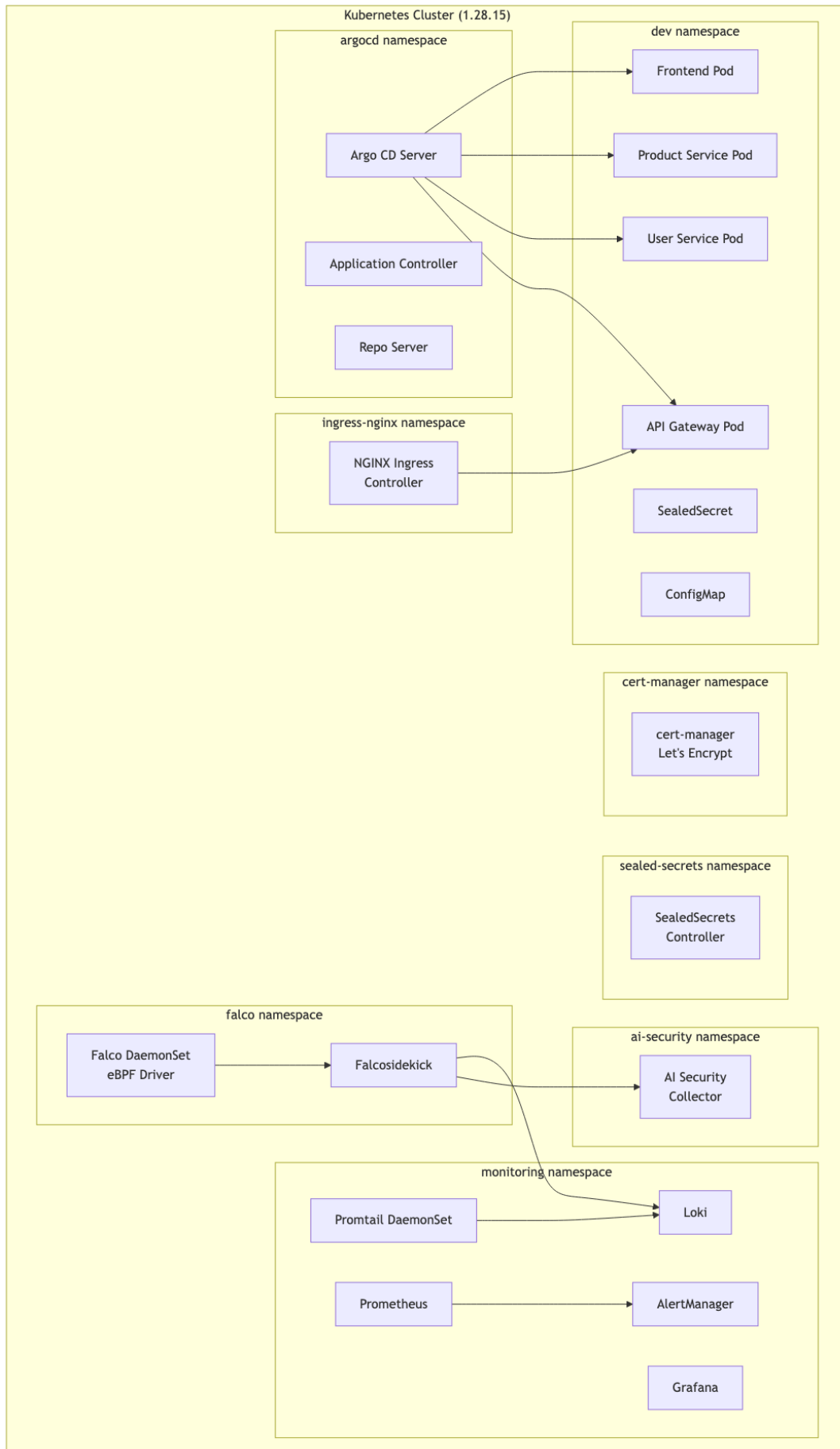


Table 7 describes the namespace allocation.

Table 7: Kubernetes namespace allocation and purpose

Namespace	Purpose	Key Resources
dev	Application workloads	Deployments, Services, SealedSecrets, ConfigMaps
monitoring	Observability stack	Prometheus, Grafana, Loki, Promtail, AlertManager
falco	Runtime security	Falco DaemonSet, Falcosidekick
sealed-secrets	Secret encryption/decryption	SealedSecrets Controller
argocd	GitOps deployment	Argo CD Server, Application Controller, Repo Server
ingress-nginx	Edge traffic routing	NGINX Ingress Controller
cert-manager	TLS certificate lifecycle	cert-manager, Let's Encrypt ClusterIssuer
ai-security	AI-augmented monitoring	AI Security Collector (FastAPI)

Argo CD GitOps configuration uses a bootstrap application pattern where a single `bootstrap-app.yaml` Application CRD recursively discovers all YAML files in the `clusters/test-cluster/` directory. This enables:

- **Self-healing:** Argo CD automatically reverts manual cluster changes to match the Git state, enforcing Git as the single source of truth.
- **Auto-pruning:** Resources deleted from the Git repository are automatically removed from the cluster.
- **Retry logic:** Failed syncs are retried with exponential backoff (up to five attempts with a maximum three-minute delay).
- **Declarative management:** Adding new infrastructure components requires only creating a YAML file in the `clusters/test-cluster/` directory.

Helm chart deployment is managed through `apps/freshbonds/`, which defines Kubernetes manifests for all four microservices. The Helm values file (`values.yaml`) centralises configuration for image tags, replica counts, resource limits, health probe settings, and security contexts. All security contexts enforce:

- `runAsNonRoot: true` - prevents root execution
- `runAsUser: 1001` - dedicated non-root user
- `readOnlyRootFilesystem: true` - prevents filesystem tampering
- `capabilities.drop: ["ALL"]` - drops all Linux capabilities
- `allowPrivilegeEscalation: false` - prevents privilege escalation

4.4 Application Layer - Microservices Design

The FreshBonds application (<https://freshbonds.com>) is a farm-to-table e-commerce platform implemented as four microservices. While the business logic is intentionally simplified (consistent with the MIS project scope), the security implementation within each service is production-grade.

Figure 4: FreshBonds microservices application architecture

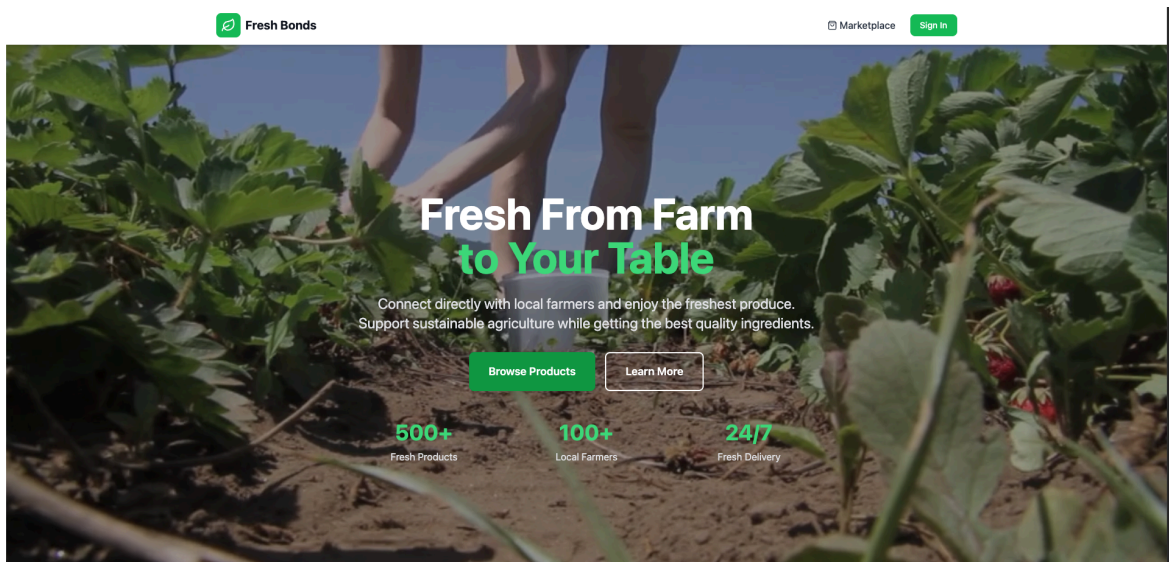
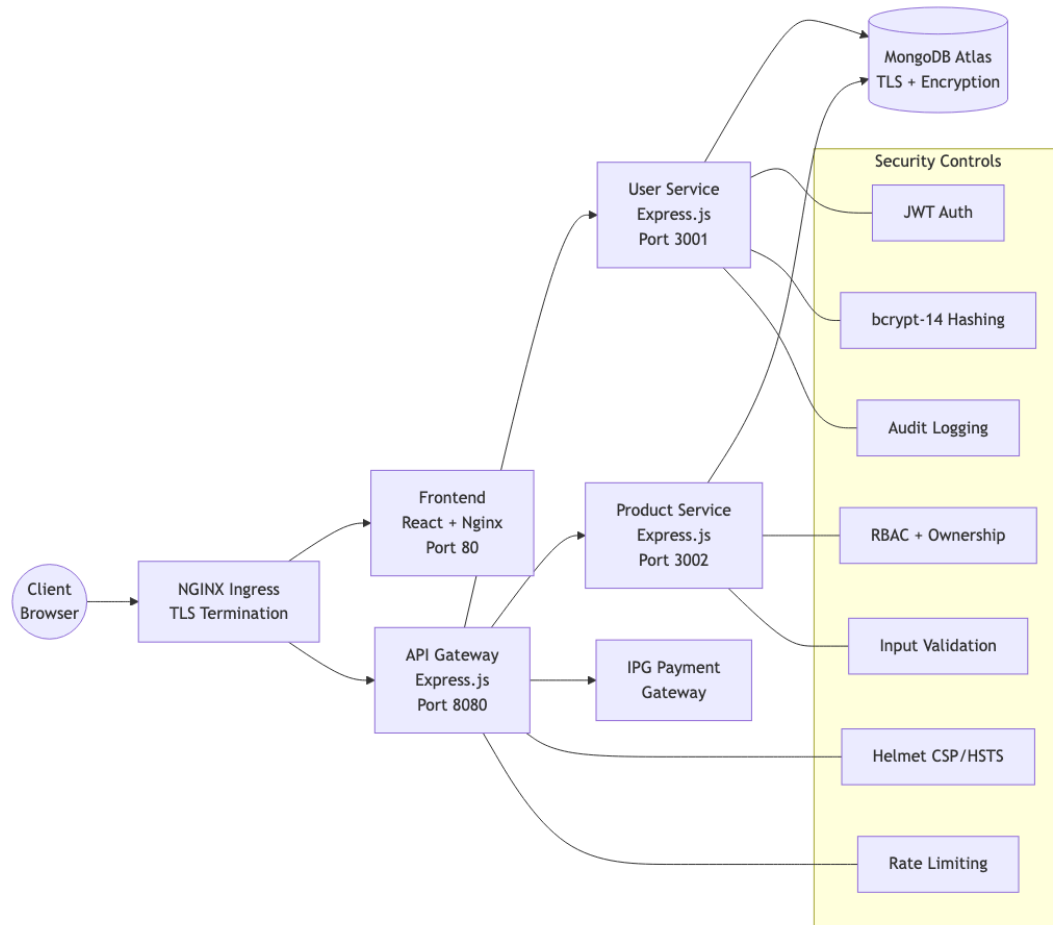


Table 8 describes the technology stack and security features for each microservice.

Table 8: Microservice technology stack and security features

Service	Technology	Port	Database	Key Security Features
API Gateway	Express.js, Helmet, http-proxy-middleware	8080	-	Content Security Policy (CSP), HSTS, X-Frame-Options, CORS origin whitelist, rate limiting (configurable per endpoint), request body size limits (1 MB), graceful shutdown with 5-second drain period, dumb-init for PID 1 signal handling
User Service	Express.js, Mongoose 8.x, bcryptjs, jsonwebtoken	3001	MongoDB Atlas	bcrypt-14 password hashing (16× stronger than default bcrypt-10), JWT with configurable expiry (8 hours default), minimum 32-character JWT secret validation at startup, anti-enumeration responses (identical error for invalid email/password), account lockout (5 failed attempts → 2-hour lockout), structured JSON audit logging capturing event type, email, result, IP address, user agent, and timestamp
Product Service	Express.js, Mongoose 8.x, validator	3002	MongoDB Atlas	Role-based access control (farmer and admin roles), ownership middleware (farmers can only modify their own products), ObjectId format validation, harvest date range validation, field whitelisting for PATCH operations, HTML escaping and regex validation
Frontend	React 18, Vite, TailwindCSS, Nginx (Alpine)	80	-	Multi-stage Docker build (node:18-alpine builder → nginx:alpine runtime), build-time environment configuration (VITE_API_URL), read-only root filesystem, non-root nginx user

Container hardening is applied uniformly across all services using a consistent Dockerfile pattern:

- **Base image:** `node:18-alpine` (minimal attack surface)
- **CVE patching:** Alpine packages updated at build time (`apk update && apk upgrade --no-cache libcrypto3 libssl3`)
- **Non-root execution:** Dedicated `nodejs` user (UID 1001, GID 1001) created and set via `USER nodejs`
- **Signal handling:** `dumb-init` installed for proper PID 1 process management, ensuring graceful shutdown on SIGTERM/SIGINT
- **Health monitoring:** Docker `HEALTHCHECK` directive configured with 30-second interval
- **Build optimisation:** Multi-stage builds to exclude build tools from the runtime image; `.dockerignore` files to prevent `node\_modules`, test files, and documentation from being copied into the image

4.5 CI/CD Pipeline Architecture

The CI/CD pipeline architecture implements the "shift left and extend right" design principle through six specialised GitHub Actions workflows that integrate ten security tools. Each pipeline addresses a specific stage of the delivery lifecycle, with automated blocking gates that prevent security violations from reaching production.

Figure 5: CI/CD pipeline architecture with security gates

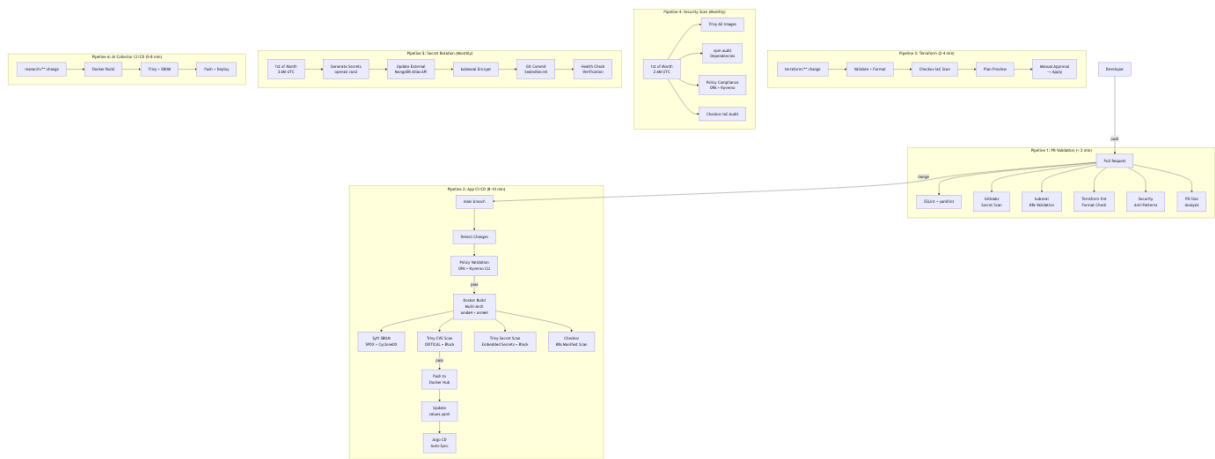


Table 9 summarises each pipeline's characteristics and integrated security tools.

Table 9: CI/CD pipeline summary and security tool integration

Pipeline	Workflow File	Trigger	Duration	Security Tools	Blocking Gates
PR Validation	pr-validation.yml	Pull requests	< 2 min	ESLint, yamllint, Gitleaks, kubeval, Terraform fmt	Secrets detected, invalid K8s manifests
App CI/CD	app-cicd.yml	Push to main/develop, tags	8–10 min	OPA/Conftest, Kyverno CLI, Trivy (CVE), Trivy (secrets), Checkov, Syft	CRITICAL CVEs, policy violations, embedded secrets
Terraform	terraform.yml	Changes to terraform/	2–4 min	Checkov, Terraform validate	IaC security violations, format errors
Security Scan	security-scan.yml	Monthly (1st @ 2 AM)	15–20 min	Trivy (all images), npm audit, OPA, Kyverno, Checkov	CRITICAL findings → GitHub Issue + PagerDuty
Secret Rotation	secret-rotation.yml	Monthly (1st @ 3 AM)	3–5 min	kubeseal, openssl	Health check failure → rollback
AI Collector CI/CD	ai-collector-cicd.yml	Push to main (research/)	5–8 min	Trivy, Syft	CRITICAL CVEs

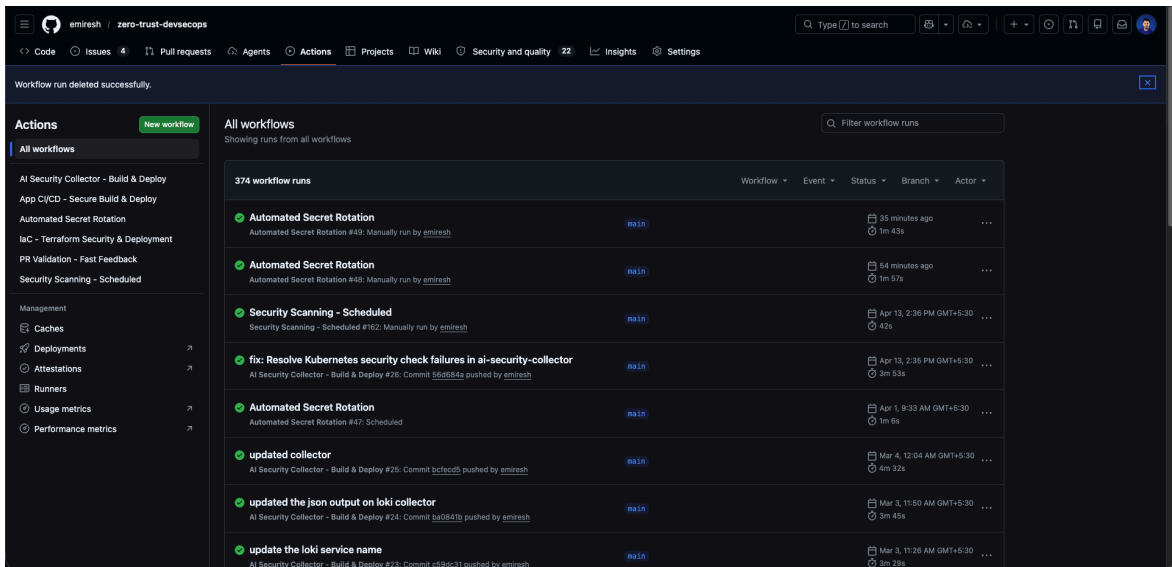
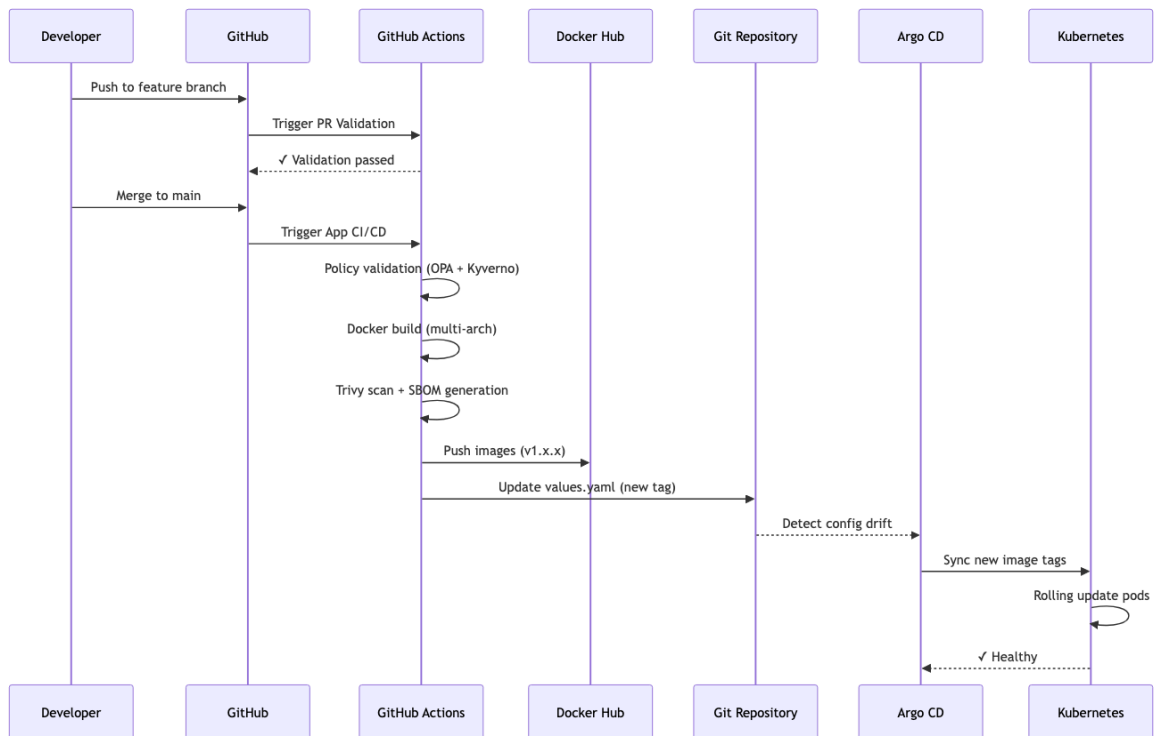


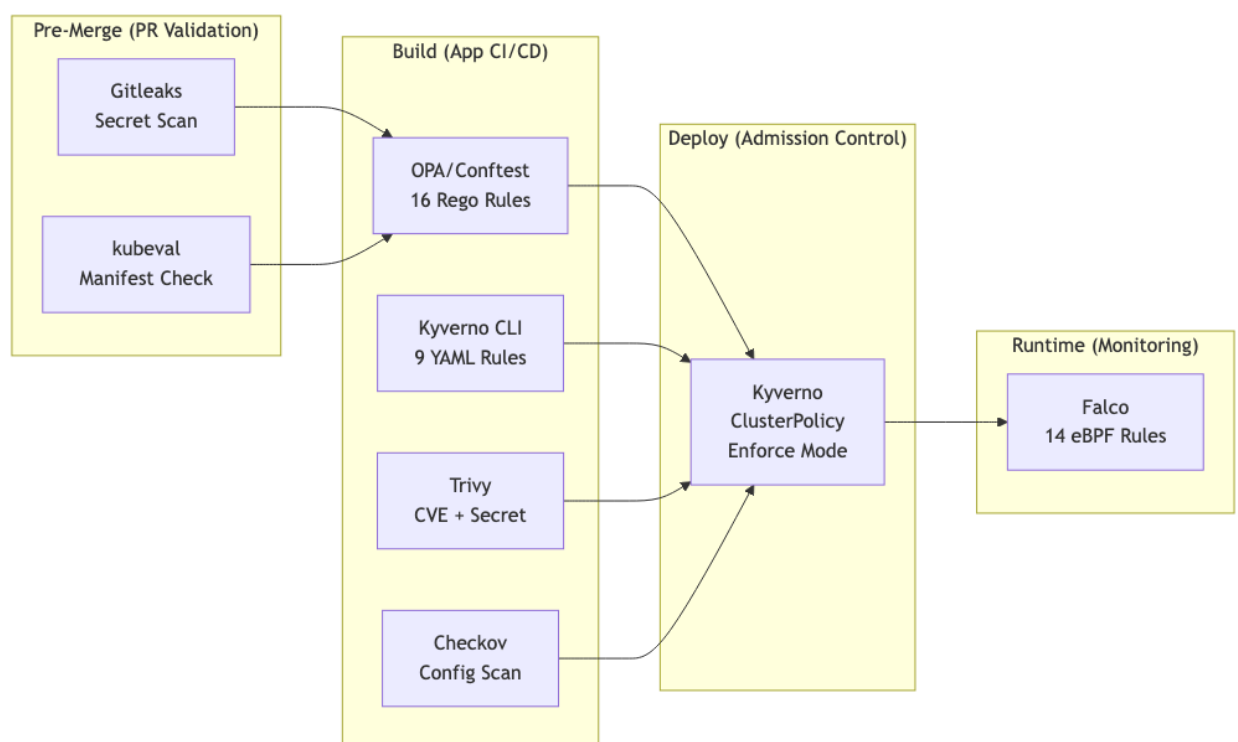
Figure 6: GitOps deployment flow via Argo CD



4.6 Policy-as-Code Implementation

The policy-as-code layer implements a dual-engine approach using Kyverno (Kubernetes-native YAML policies) and Open Policy Agent (general-purpose Rego policies). This dual-engine design provides complementary coverage: Kyverno excels at Kubernetes-specific resource validation with a shallow learning curve, while OPA enables complex cross-resource policies and is applicable beyond Kubernetes contexts.

Figure 7: Policy-as-code enforcement points across the lifecycle



Kyverno policies (defined in `policies/kyverno/`) consist of nine ClusterPolicy rules in enforce mode:

13. **run-as-non-root** - enforces `securityContext.runAsNonRoot: true` on all Deployments in production and dev namespaces.
14. **disallow-privileged** - prevents `securityContext.privileged: true` on all workloads.

15. **require-resource-limits** - mandates both CPU and memory limits on all containers.
16. **drop-all-capabilities** - requires `capabilities.drop: ["ALL"]` on all containers.
17. **disallow-privilege-escalation** - enforces `allowPrivilegeEscalation: false`.
18. **read-only-root-filesystem** - requires `readOnlyRootFilesystem: true` for production workloads.
19. **disallow-latest-tag** - prevents the use of `:latest` image tags, requiring explicit semantic versioning.
20. **require-approved-registry** - restricts image sources to approved registries: `docker.io`, `ghcr.io`, and `gcr.io`.
21. **require-image-pull-policy** - enforces explicit `imagePullPolicy` of `Always` or `IfNotPresent`.

OPA policies (defined in `policies/opa/`) consist of sixteen deny and warn rules expressed in the Rego language:

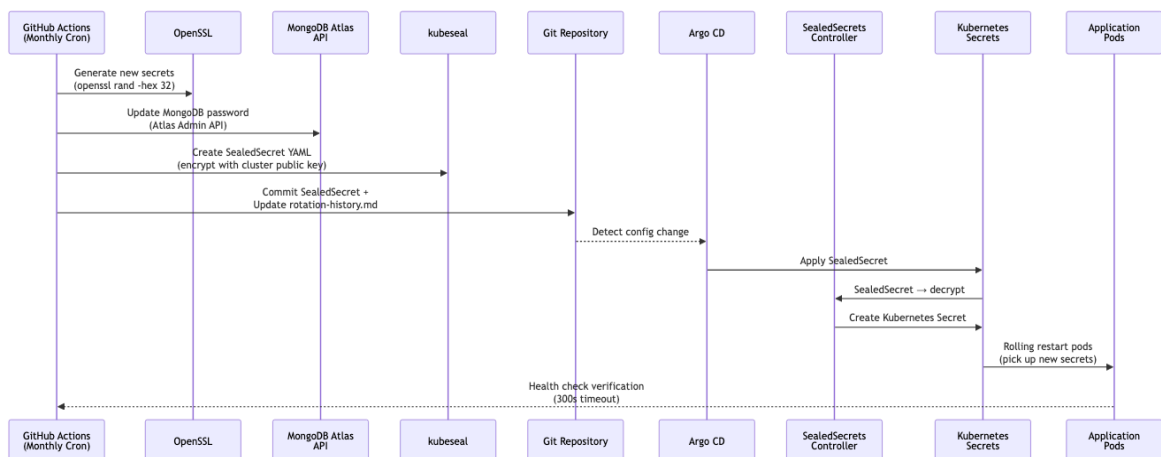
- **Security rules (13):** `runAsNonRoot` (deny), no privileged containers (deny), resource limits (deny), readiness probe (warn), liveness probe (warn), no privilege escalation (deny), no latest tag (deny), approved registry (deny), no `hostPath` volumes (deny), read-only root (warn), no dangerous capabilities (deny), drop ALL capabilities (deny), and environment variable validation.
- **Network rules (3):** `NetworkPolicy` required per namespace (deny), `LoadBalancer` service warning (warn), and missing service selector warning (warn).

Both policy engines are validated in the CI/CD pipeline through the App CI/CD workflow, where `conftest test` (for OPA) and `kyverno apply` (for Kyverno CLI) are executed against all Kubernetes manifests. Policy violations at this stage block the deployment pipeline.

4.7 Secret Lifecycle Management

Secret management implements a complete lifecycle from generation through rotation, with Git-based audit trails providing compliance evidence. The system uses Bitnami Sealed Secrets to enable GitOps-compatible secret storage - encrypted secrets are committed to Git and can only be decrypted by the Sealed Secrets controller running within the target cluster using a private key that never leaves the cluster.

Figure 8: Secret lifecycle management and rotation workflow



Secrets rotated monthly:

- **MongoDB password:** 32-character random string generated via `openssl rand`, updated through the MongoDB Atlas Admin API, sealed with `kubeseal`, and committed as a `SealedSecret`.
- **JWT signing key:** 64-character hex string generated via `openssl rand -hex 32`, updating the application environment variable. By design, this invalidates all existing JWT tokens, forcing re-authentication - a deliberate Zero Trust control.
- **API tokens:** IPG payment gateway application token and callback authentication token.

Audit trail: Every rotation is logged in `docs/rotation-logs/rotation-history.md` with timestamp, secrets rotated, verification status, and operator (automated or manual). Git history provides an immutable audit trail for compliance evidence.

4.8 Runtime Security - Falco and eBPF

Falco is deployed as a Kubernetes DaemonSet in the `falco` namespace, using the modern eBPF driver (rather than the legacy kernel module) for kernel-level system call monitoring. The eBPF driver operates with minimal performance overhead and does not require kernel module compilation, making it suitable for cloud environments where kernel access may be restricted.

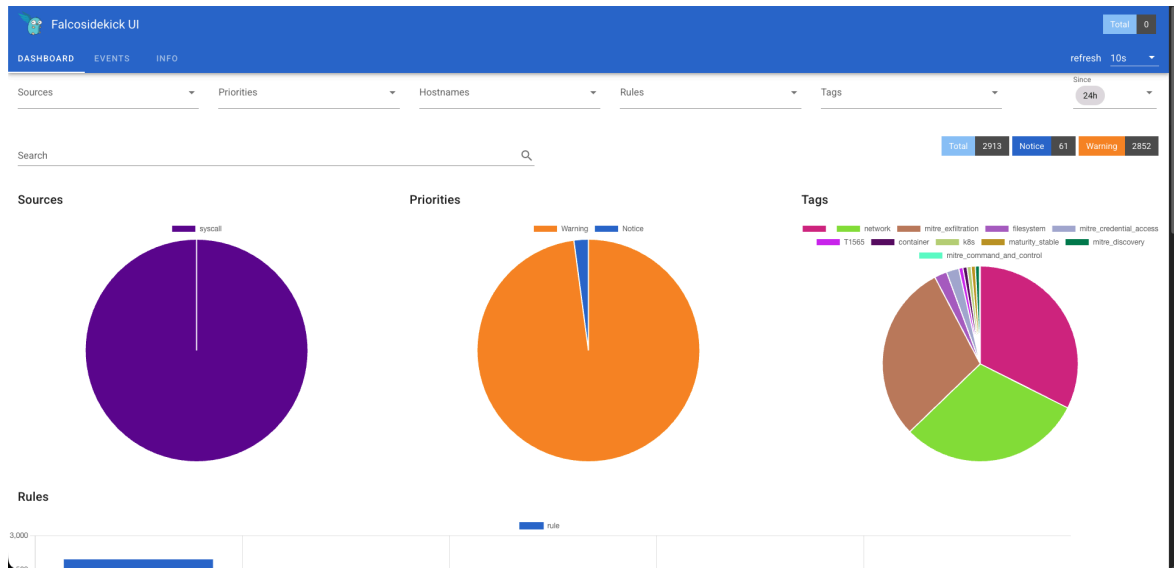
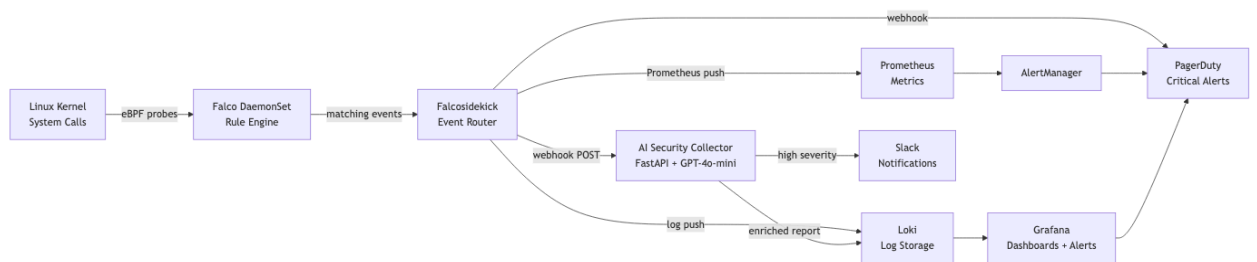


Figure 9: Runtime security event flow from Falco to AI enrichment



Thirteen custom Falco rules are implemented, each mapped to a MITRE ATT&CK technique (detailed mapping in Appendix C). The rules cover:

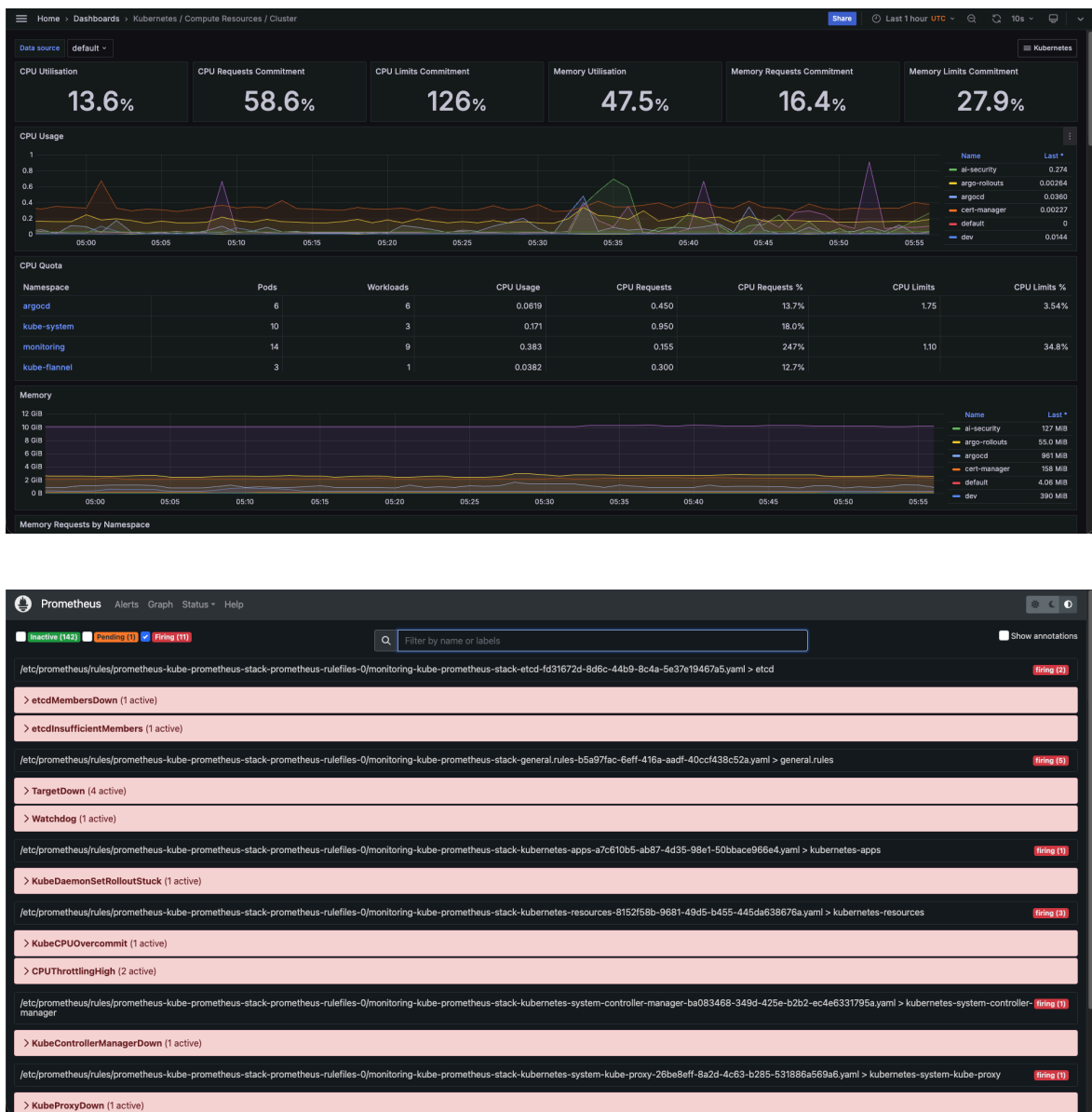
22. **Shell Spawned in Container** (WARNING) - Detects `bash`, `sh`, `zsh`, `dash`, or `ksh` execution within containers. Maps to MITRE T1059 (Command-Line Interface).
23. **Package Management in Container** (WARNING) - Detects `apt`, `yum`, `pip`, or `npm` execution. Maps to MITRE T1546 (Event Triggered Execution).

24. **Detect Cryptocurrency Mining** (CRITICAL) - Detects xmrig, minerd, or stratum protocol connections. Maps to MITRE T1496 (Resource Hijacking).
25. **Read Sensitive File** (WARNING) - Detects reads of `/etc/shadow`, `/etc/sudoers`, `.ssh/*`, `.aws/*`, `.kube/*`. Maps to MITRE T1552 (Unsecured Credentials).
26. **Privilege Escalation via Setuid** (CRITICAL) - Detects `setuid` system calls setting UID to 0. Maps to MITRE T1548 (Abuse Elevation Control Mechanism).
27. **Reverse Shell Detection** (CRITICAL) - Detects network-connected shell processes. Maps to MITRE T1059.004 (Unix Shell).
28. **Container Escape Attempt** (CRITICAL) - Detects `nsenter`, `docker`, `runc`, or `crictl` execution within containers. Maps to MITRE T1611 (Escape to Host).
29. **Outbound Suspicious Port** (WARNING) - Detects connections to known attacker ports (4444, 5555, 6666, 1337). Maps to MITRE T1571 (Non-Standard Port).
30. **Suspicious File Modification** (CRITICAL) - Detects writes to `/etc/passwd`, `/etc/shadow`, `crontab`, or `authorized_keys`. Maps to MITRE T1543 (Create or Modify System Process).
31. **Network Reconnaissance** (WARNING) - Detects `nmap`, `masscan`, or `zmap` execution. Maps to MITRE T1046 (Network Service Scanning).
32. **Suspicious DNS Query** (WARNING) - Detects DNS queries containing `tor`, `proxy`, or VPN domains. Maps to MITRE T1071.004 (Application Layer Protocol: DNS).
33. **Large Data Transfer** (WARNING) - Detects outbound transfers exceeding 10 MB. Maps to MITRE T1048 (Exfiltration over Alternative Protocol).
34. **Read Sensitive File Untrusted** (WARNING) - Detects sensitive file access by non-trusted containers. Maps to MITRE T1552.001 (Credentials in Files).

4.9 Observability Stack

The observability layer implements a three-pillar monitoring approach: metrics (Prometheus), logs (Loki), and alerts (AlertManager and Grafana).

Figure 10: Grafana, Prometheus, AlertManager, Slack, PagerDuty dashboards



AlertmanagerAlertsSilencesStatusSettingsHelp

New Silence

FilterGroup

Receiver: AllSilencedInhibited

Custom matcher, e.g. env="production"

+

Silence

Expand all groups

+ pagerduty-notifications

cluster="test-cluster"

+

363 alerts

+ pagerduty-notifications

alertname="CPUThrottlingHigh"

+

1 alert

+ pagerduty-notifications

alertname="KubeCPUOvercommit"

+

1 alert

+ pagerduty-critical

alertname="KubeControllerManagerDown"

+

1 alert

+ pagerduty-critical

alertname="KubeProxyDown"

+

1 alert

+ pagerduty-critical

alertname="KubeSchedulerDown"

+

1 alert

+ null

alertname="Watchdog"

+

1 alert

+ pagerduty-notifications

alertname="KubeDaemonSetRolloutStuck"

+

service="kube-prometheus-stack-kube-state-metrics"

+

1 alert

+ pagerduty-critical

alertname="PrometheusRuleFailures"

+

service="kube-prometheus-stack-prometheus"

+

2 alerts

+ pagerduty-notifications

alertname="TargetDown"

+

service="kube-prometheus-stack-kube-controller-manager"

+

1 alert

HomeThreadsHuddlesDirectories

Warning Security Event Detected

Rule: Read sensitive file untrustedContainer: Namespace: Priority: Warning

+ 20+ new messages

PagerDutyIncidentsServicesPeopleAutomationAIOpsAnalyticsIntegrationsStatusAI

Search

🔔📧👤

Your open incidents (Last 30 days)

5 triggered0 acknowledged

All open incidents (Last 30 days)

5 triggered0 acknowledged

AcknowledgeReassignResolveSnooze

Go to incident #...

OpenTriggeredAcknowledgedResolvedAny Status

Assigned to meAllLast 30 days

Status	Priority	Urgency	Type	Title	Created	Service	Assigned To
Triggered	--	High	Base Incident	DaemonSet rollout is stuck. [SHOW DETAILS (1 triggered alert)]	#3556 at 11:20 AM	Kubernetes Infrastructure	Iresh Ekanayaka
Triggered	--	High	Base Incident	Processes experience elevated CPU throttling. [SHOW DETAILS (1 triggered alert)]	#3555 at 11:18 AM	Kubernetes Infrastructure	Iresh Ekanayaka
Triggered	--	High	Base Incident	Large Data Transfer [SHOW DETAILS (1 triggered alert)]	#3554 at 11:02 AM	Kubernetes Infrastructure	Iresh Ekanayaka

Per Page: 251-3

Activity over last 7 days

Service	Title	Time	Activity
Kubernetes Infrastructure	[#3556] DaemonSet rollout is stuck.	at 11:20 AM	Triggered through the API. Description: DaemonSet rollout is stuck. (View Message)
Kubernetes Infrastructure	[#3555] Processes experience elevated CPU throttling.	at 11:18 AM	Triggered through the API. Description: Processes experience elevated CPU throttling. (View Message)

New Incident

You're always on-call

You are always on-call for

DevOps

Lt

ON CALL NOW

DevOps

LEVEL 1

Iresh Ekanayaka

on-call all the time

GETTING STARTED

Setting up PagerDuty

How do I create my first notification?

PD University

INFO

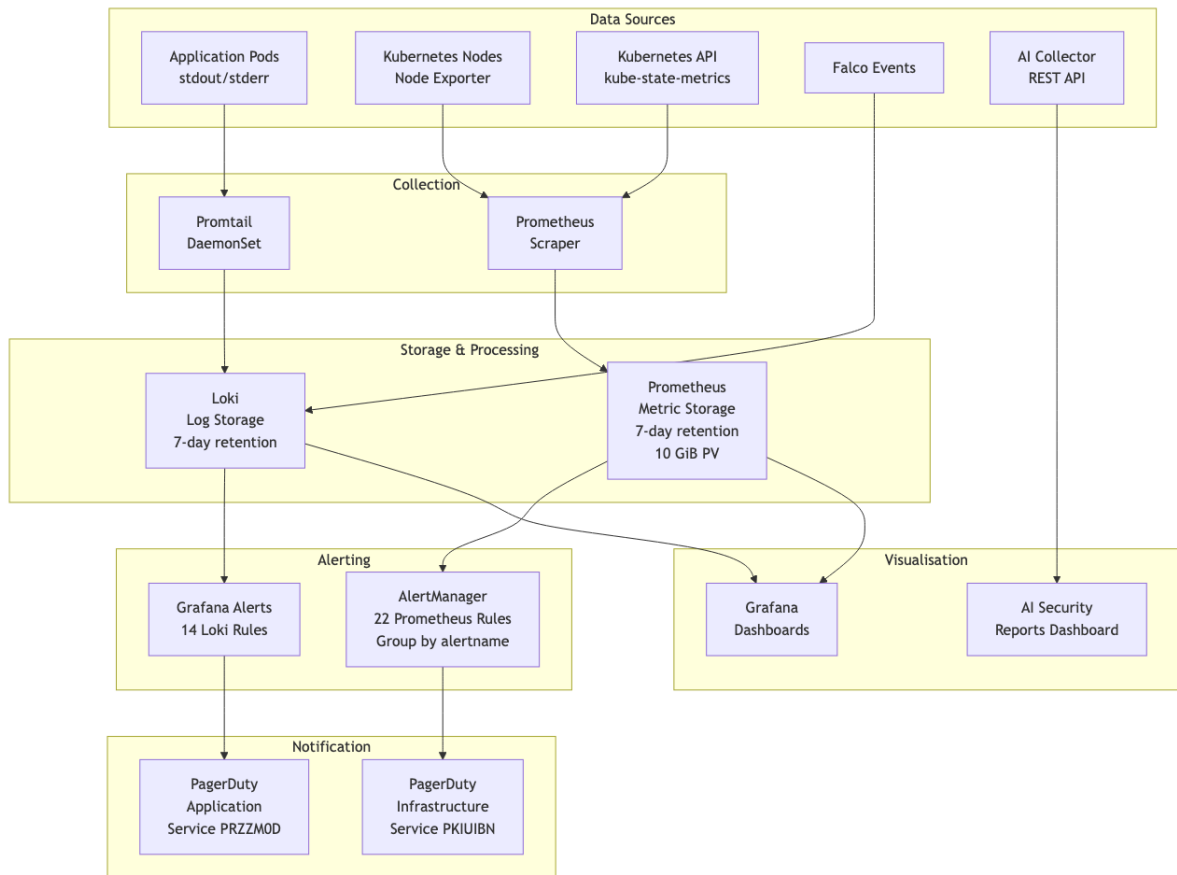
What's new in PagerDuty?

PagerDuty System Status

Get the PagerDuty Mobile App

?

Figure 11: Observability stack data flow



Prometheus alert rules (22 rules) are organised across three categories:

- **Infrastructure alerts (6):** DiskSpaceWarning (80%), DiskSpaceCritical (95%), NodeMemoryPressure, NodeDiskPressure, PVCAAlmostFull (90%), NodeNotReady.
- **Application alerts (10):** ServiceDown, HighMemoryUsage (>90%), HighCPUUsage (>90%), PodCrashLooping, ContainerOOMKilled, ServiceReplicas mismatch, ServiceAvailability (high error rate), MongoDBConnectionFailures, APIGatewayHighLatency, PaymentProcessingFailures.
- **Falco monitoring (3):** FalcoDown, FalcoHighEventRate (>1000/min), FalcoDroppedEvents.

AlertManager routing groups alerts by alertname, cluster, and service, with a two-tier routing strategy:

- **Critical alerts** → PagerDuty Infrastructure service (PKIUIBN) with immediate escalation.
- **Warning and informational alerts** → PagerDuty notifications with standard escalation policy.

Grafana Loki alert rules (14 rules) provide log-based alerting in three categories:

- **Application alerts (5):** High error rate, crash detection (CrashLoopBackOff), MongoDB connection failures, 5xx HTTP errors, payment processing failures.
- **Log stream alerts (5):** Pod log error rate, MongoDB disconnection patterns, payment service unavailability, authentication failure rate spike, pod restart loop detection.
- **Login security alerts (4):** Brute force detection (failed login spike), single-user targeting (multiple failed attempts on one account), suspicious IP patterns, email enumeration attempts (registration failures).

4.10 AI-Augmented Security Monitoring

The AI Security Collector is a FastAPI-based Python service deployed in the `ai-security` namespace that receives Falco security events from Falcosidekick via webhook and enriches them with contextual threat analysis generated by Azure OpenAI GPT-4o-mini.

Architecture: Falcosidekick routes matched Falco events as HTTP POST requests to the AI Collector's `/events` endpoint. The collector:

35. **Parses** the incoming Falco event, extracting priority, rule name, output fields, container metadata, and timestamp.
36. **Deduplicates** events using a configurable window (default 60 seconds) to prevent excessive API calls for repeated events.
37. **Queries Prometheus** for contextual metrics - container CPU usage, memory consumption, and restart rate - to enrich the event with operational context.
38. **Generates an AI report** by sending a structured prompt to Azure OpenAI GPT-4o-mini (temperature=0.3 for deterministic output) requesting:

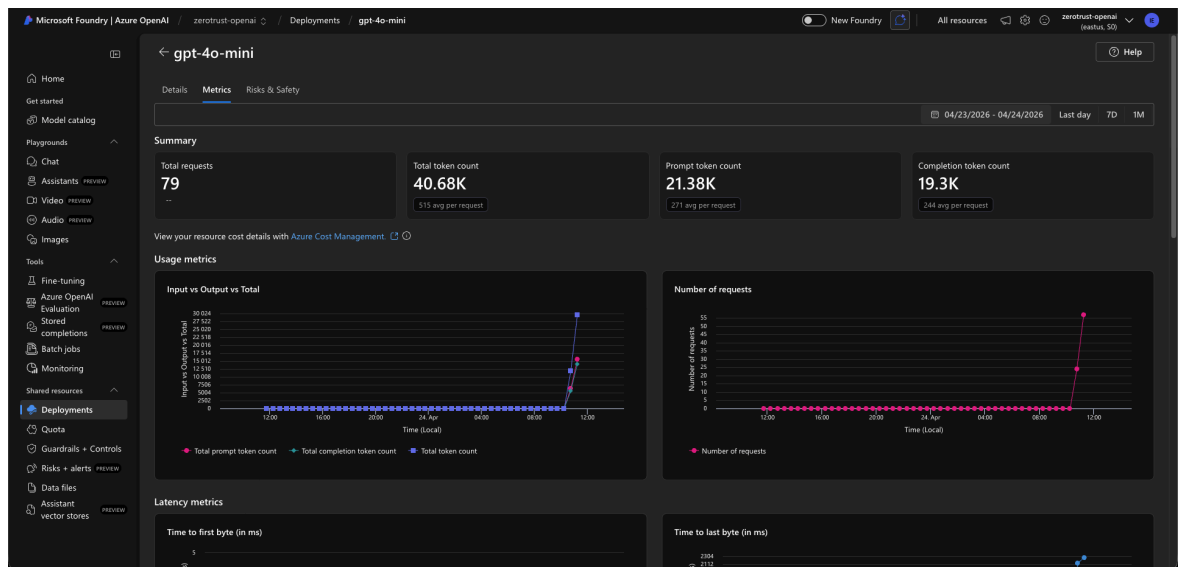
- Threat assessment with severity rating and confidence level
- Investigation steps for the security analyst
- Recommended remediation actions
- MITRE ATT&CK technique mapping

39. **Stores** the enriched report in Loki (primary) or JSONL files (fallback if Loki is unavailable).

40. **Routes** high-severity reports to Slack and PagerDuty for immediate attention.

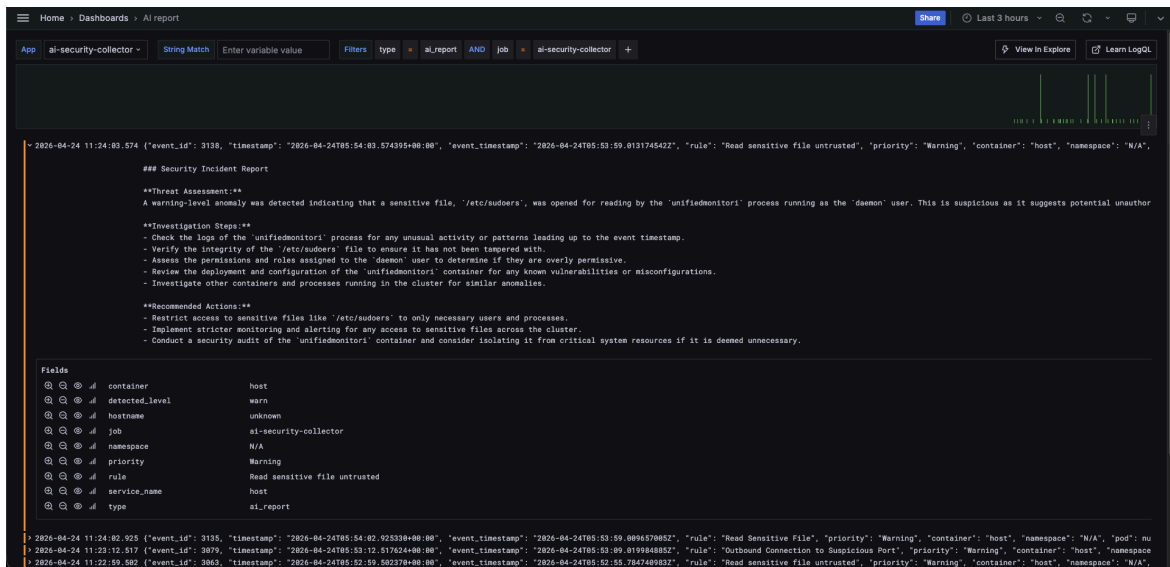
Key design decisions:

- AI enrichment is **non-blocking** - it does not sit in the critical alerting path. Raw Falco events flow directly to Loki, Prometheus, and PagerDuty through Falcosidekick. The AI Collector provides additional context on a parallel path, ensuring that alerting is never delayed or blocked by API latency or LLM unavailability.
- **Deduplication** prevents API cost escalation during event storms (e.g., repeated shell spawns during debugging sessions).
- The `/health` and `/stats` endpoints provide operational visibility into the collector's status, event counts, and API usage.



Grafana AI Security Reports dashboard visualises enriched reports with panels for:

- AI Reports by Priority (donut chart)
- Recent AI Security Reports (colour-coded severity table)
- AI Reports Timeline (stacked time-series trend)
- Top Triggered Falco Rules (bar chart)
- Latest AI Report Full Analysis (markdown panel with GPT-4o-mini output)
- AI Collector Health (status indicator)



4.11 Implementation Artefacts Summary

Table 11 provides a comprehensive summary of implementation artefacts with their repository locations and line counts, serving as verifiable evidence of the implementation scope.

Table 11: Implementation artefact summary with repository evidence

Category	Artefact	Repository Path	Key Metrics
Infrastructure	Terraform definitions	terraform/.tf	12 files; OCI VCN, 3 instances, security lists
Kubernetes	Cluster manifests	clusters/test-cluster//.yaml	22+ YAML files; 8 namespaces
Application	Microservices source	src//	4 services; ~6,612 LOC (JavaScript/JSX)
Helm	Application chart	apps/freshbonds/	values.yaml + deployment templates
CI/CD	GitHub Actions workflows	.github/workflows/.yaml	6 pipelines; ~2,837 lines of YAML
Policy	Kyverno policies	policies/kyverno/.yaml	9 rules (enforce mode)
Policy	OPA/Rego policies	policies/opa/.rego	16 deny/warn rules
Runtime	Falco custom rules	clusters/test-cluster/05-infrastructure/falco.yaml	13 custom rules + macros
Monitoring	Prometheus rules	clusters/test-cluster/05-infrastructure/rules.yaml	22 alert rules
AI Collector	Python FastAPI service	research/	FastAPI + Azure OpenAI integration
Secrets	Rotation workflow	.github/workflows/secret-rotation.yaml	Monthly rotation with audit trail
GitOps	Argo CD bootstrap	bootstrap/bootstrap-app.yaml	Recursive auto-sync + self-heal
Documentation	Project docs	docs/	50+ files across 6 categories

5. Evaluation

5.1 Evaluation Setup and Methodology

The evaluation uses repository investigation as the primary evidence method, aligning with the design science evaluation approach recommended by Hevner *et al.* (2004). Rather than relying solely on theoretical claims, each evaluation criterion is verified by examining the actual artefacts - workflow files, policy definitions, Kubernetes manifests, Helm values, and pipeline execution logs - within the `zero-trust-devsecops` Git repository.

Evaluation experiments and evidence sources:

#	Evaluation Criterion	Evidence Source	Verification Method
E1	NIST SP 800-207 compliance	All repository artefacts	Map each tenet to specific files, configurations, and controls
E2	Pipeline gate effectiveness	.github/workflows/.yaml	Verify each gate's trigger condition, action, and blocking behaviour
E3	Policy enforcement coverage	policies/kyverno/.yaml, policies/opa/.rego, Falco rules	Count rules by security domain and enforcement type
E4	Runtime detection capability	Falco custom rules, MITRE ATT&CK mapping	Assess technique coverage and expected detection latency
E5	Supply chain integrity	App CI/CD workflow, SBOM outputs, secret rotation workflow	Verify scanning coverage, SBOM formats, and rotation audit trail

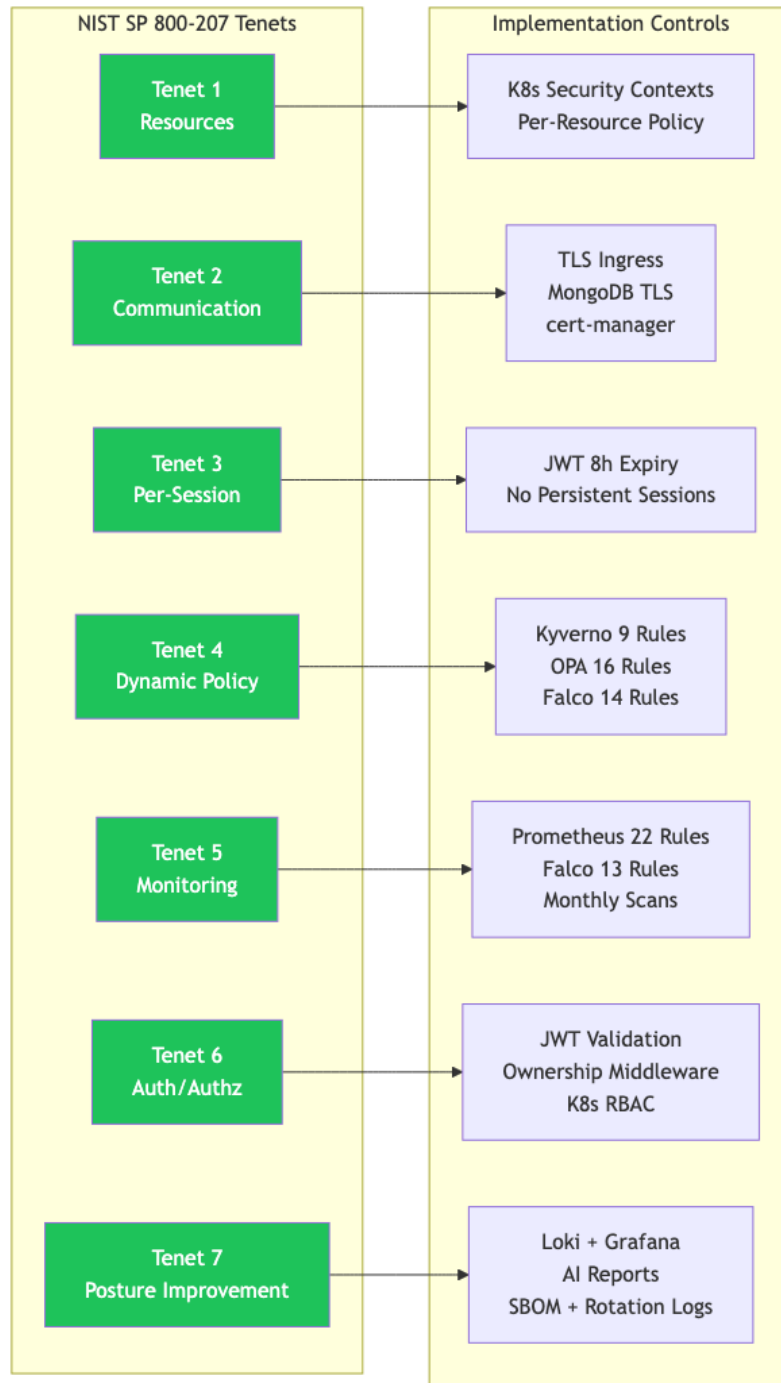
5.2 NIST SP 800-207 Zero Trust Compliance Mapping

Table 12 maps each of the seven NIST SP 800-207 Zero Trust Architecture tenets to the specific implementation controls within the project, with an assessment of compliance strength.

Table 12: NIST SP 800-207 Zero Trust tenet compliance mapping

#	NIST ZTA Tenet	Implementation Evidence	Assessment
1	All data sources and computing services are considered resources	Every component - pods, services, databases, secrets, images, infrastructure, and workflows - is individually secured with its own security context, credentials, and access policy. Kubernetes manifests in clusters/test-cluster/ and apps/freshbonds/ define per-resource security.	✓ Strong
2	All communication is secured regardless of network location	TLS termination at ingress via cert-manager and Let's Encrypt (clusters/test-cluster/05-infrastructure/cert-manager.yaml). MongoDB Atlas requires TLS for all connections. Internal services addressed through Kubernetes ClusterIP services. Security lists restrict network ingress.	✓ Strong
3	Access to individual enterprise resources is granted on a per-session basis	JWT tokens with 8-hour expiry issued by the User Service. No persistent sessions; each API request requires a valid token. Token verified against database on every request. Product Service enforces per-request ownership validation.	✓ Strong
4	Access to resources is determined by dynamic policy	Kyverno (9 rules) + OPA (16 rules) + Falco (14 rules) enforce policy dynamically across admission control, CI/CD, and runtime. Policies are version-controlled in policies/ and can be updated without service restart. Argo CD automatically applies policy changes.	✓ Strong
5	The enterprise monitors and measures the integrity and security posture of all owned and associated assets	Prometheus metrics (22 alert rules), Falco runtime monitoring (13 custom rules), Trivy image scanning, Checkov IaC scanning, npm dependency auditing, and monthly security scan workflow provide continuous monitoring across all asset types.	✓ Strong
6	All resource authentication and authorisation are dynamic and strictly enforced before access is allowed	JWT validation on every API request (src/user-service/), ownership middleware verifying resource access (src/product-service/), Kubernetes RBAC, SealedSecrets requiring cluster private key for decryption, and security context enforcement on all pods.	✓ Strong
7	The enterprise collects as much information as possible about the current state of assets, network infrastructure, and communications and uses it to improve its security posture	Three-flow observability (metrics via Prometheus, logs via Loki, security events via Falco). AI enrichment provides contextual threat analysis. Monthly security audits via scheduled workflow. Secret rotation audit trails in docs/rotation-logs/rotation-history.md. SBOM artefacts provide supply chain visibility.	✓ Strong

Figure 12: NIST SP 800-207 compliance mapping visualisation



5.3 Pipeline Security Gate Effectiveness

Table 13 evaluates each security gate in the CI/CD pipeline architecture, specifying the trigger condition, expected action, and blocking behaviour.

Table 13: Pipeline security gate effectiveness

Pipeline	Gate	Tool	Trigger Condition	Action	Blocking
PR Validation	Secret scan	Gitleaks	Secrets detected in code diff	Fail security check	Yes - blocks merge
PR Validation	K8s validation	kubeval	Invalid Kubernetes manifest syntax	Report validation failure	Yes - blocks merge
PR Validation	Code quality	ESLint, yamllint	Linting violations detected	Report warnings/errors	Yes - blocks merge
PR Validation	Format check	Terraform fmt	Unformatted Terraform code	Fail format check	Yes - blocks merge
App CI/CD	Policy validation	OPA/Conftest	Rego policy violation in manifests	Block deployment pipeline	Yes - blocks deploy
App CI/CD	Policy validation	Kyverno CLI	YAML policy violation in manifests	Block deployment pipeline	Yes - blocks deploy
App CI/CD	CVE scan	Trivy	CRITICAL vulnerability count > 0	Block image push	Yes - blocks deploy
App CI/CD	Secret scan	Trivy (secret mode)	Embedded secrets found in built image	Block image push	Yes - blocks deploy
App CI/CD	Config scan	Checkov	Kubernetes manifest misconfiguration	Report SARIF findings	Yes - blocks deploy
App CI/CD	SBOM generation	Syft	Image build completed	Produce SPDX + CycloneDX artefacts	No (generates artefact)
Terraform	IaC scan	Checkov	Infrastructure security violation	Block Terraform apply	Yes - blocks apply
Terraform	Manual gate	-	Production infrastructure change	Require human approval	Yes - blocks apply
Security Scan	Image scan	Trivy	CRITICAL/HIGH findings in production images	Create GitHub Issue + PagerDuty alert	Yes - creates alert
Security Scan	Dependency scan	npm audit	CRITICAL/MODERATE dependency CVE	Report findings	Yes - creates alert
Secret Rotation	Health check	-	Post-rotation service unavailability	Alert and rollback	Yes - triggers alert

To validate CI/CD security gate effectiveness, a container image containing a known CRITICAL vulnerability was intentionally introduced during the build stage. The Trivy vulnerability scanner identified the issue and triggered a pipeline failure, preventing deployment to the Kubernetes cluster. This confirms that the implemented security gates effectively block insecure workloads before reaching production environments.

5.4 Policy Enforcement Coverage Analysis

Table 14 analyses the distribution of policy rules across security domains, showing the complementary coverage provided by the three policy engines.

Table 14: Policy enforcement coverage by security domain

Security Domain	Kyverno Rules	OPA Rules	Falco Rules	Total	Enforcement Point
Container Privilege	3	3	2	8	Admission + CI/CD + Runtime
Resource Management	1	1	0	2	Admission + CI/CD
Image Security	3	2	0	5	Admission + CI/CD
Filesystem Security	1	1	2	4	Admission + Runtime
Capability Restrictions	1	2	0	3	Admission + CI/CD
Network Security	0	3	3	6	CI/CD + Runtime
Credential Protection	0	0	2	2	Runtime
Health Monitoring	0	2	0	2	CI/CD
Execution Control	0	0	3	3	Runtime
Data Protection	0	0	2	2	Runtime
Total	9	16	14	39	

The analysis reveals that the dual-engine CI/CD validation (Kyverno + OPA) provides strong coverage for deployment-time controls, while Falco provides complementary runtime coverage for threat categories that cannot be detected through static analysis - execution control, credential access, network reconnaissance, and data exfiltration.

Policy enforcement was validated by deploying a non-compliant Kubernetes workload configured with privileged container access and missing resource limits. Both Kyverno and OPA policies successfully rejected the deployment during validation, demonstrating effective enforcement of security constraints at the admission control level.

5.5 Runtime Detection Capability Assessment

Table 15 assesses the detection capability of each custom Falco rule, including the threat category, detection method, and expected detection latency.

Table 15: Runtime detection capability assessment

Threat Category	Custo m Rules	Detection Method	Expected Latency
Container shell access	1	Process exec monitoring via eBPF	< 1 second
Unauthorised package installation	1	Process exec monitoring via eBPF	< 1 second
Cryptocurrency mining	1	Process name + network monitoring	< 5 seconds
Sensitive file access	2	File descriptor monitoring via eBPF	< 1 second
Privilege escalation	2	Syscall + capability monitoring	< 1 second
Reverse shell	1	Network + process monitoring	< 5 seconds
Container escape	1	Process + namespace monitoring	< 1 second
C2 communication	2	Network connection monitoring	< 5 seconds
Filesystem tampering	1	File write monitoring via eBPF	< 1 second
Network reconnaissance	1	Process exec monitoring via eBPF	< 1 second
Data exfiltration	1	Network transfer size monitoring	< 30 seconds

The eBPF-based detection mechanism operates at the kernel level, providing sub-second detection latency for most threat categories. The 30-second upper bound for data exfiltration detection reflects the threshold-based approach (>10 MB transfer), which inherently requires accumulation before triggering.

To evaluate runtime threat detection capability, a simulated security test was conducted by executing a controlled suspicious activity within a Kubernetes container. A test pod was deployed and a modification was made to the `/etc/hosts` file, representing unauthorised file tampering behaviour commonly associated with container compromise.

This activity triggered a **CRITICAL** alert from the Falco runtime security engine, which identified the behaviour as a suspicious system-level modification. The alert was forwarded via Falcosidekick to the AI Security Collector service, where the event was successfully received and processed. When AI enrichment was enabled, the system generated contextual analysis for the detected event.

This experiment confirms that the framework is capable of detecting and processing runtime security events in near real-time, demonstrating effective monitoring and response capabilities within the Kubernetes environment.

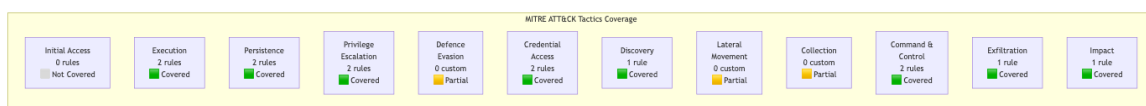
5.6 MITRE ATT&CK Coverage Analysis

Table 16 maps the custom Falco rules to the MITRE ATT&CK for Containers framework, identifying coverage across tactics and techniques.

Table 16: MITRE ATT&CK for Containers coverage analysis

MITRE ATT&CK Tactic	Techniques Covered	# Rules	Coverage Assessment
Initial Access	-	0	Not covered (external boundary; outside container scope)
Execution	T1059 (Command-Line Interface), T1059.004 (Unix Shell)	2	Shell spawn, reverse shell detection
Persistence	T1546 (Event Triggered Execution), T1543 (Create/Modify System Process)	2	Package install, critical file modification
Privilege Escalation	T1548 (Abuse Elevation Control), T1611 (Escape to Host)	2	Setuid detection, container escape attempt
Defence Evasion	-	0	Partially covered by built-in Falco rules
Credential Access	T1552 (Unsecured Credentials), T1552.001 (Credentials in Files)	2	Sensitive file reads by trusted/untrusted containers
Discovery	T1046 (Network Service Scanning)	1	Reconnaissance tool detection (nmap, masscan)
Lateral Movement	-	0	Covered by NetworkPolicy and network segmentation
Collection	-	0	Partially covered by file monitoring rules
Command and Control	T1571 (Non-Standard Port), T1071.004 (DNS)	2	Suspicious port connections, DNS tunnelling
Exfiltration	T1048 (Exfiltration over Alternative Protocol)	1	Large data transfer detection (>10 MB)
Impact	T1496 (Resource Hijacking)	1	Cryptocurrency mining detection

Figure 13: MITRE ATT&CK for Containers coverage heat map



Coverage summary: Custom Falco rules cover 13 techniques across 8 of the 12 MITRE ATT&CK tactics. Initial Access is out of scope (external to containers), while Defence Evasion, Lateral Movement, and Collection are partially covered through built-in Falco rules and network policies. The strongest coverage is in Execution, Privilege Escalation, Credential Access, and Command and Control - the tactics most commonly observed in container compromise incidents.

5.7 Supply Chain Security Assessment

Table 17: Supply chain security control assessment

Control	Implementation	Format/Standard	Evidence
Container vulnerability scanning	Trivy in App CI/CD and monthly Security Scan	SARIF + JSON reports	.github/workflows/app-cicd.yml
Embedded secret scanning	Trivy secret mode in App CI/CD	JSON report	.github/workflows/app-cicd.yml
SBOM generation	Syft in App CI/CD and AI Collector CI/CD	SPDX (ISO/IEC 5962:2021) + CycloneDX (OWASP)	.github/workflows/app-cicd.yml
Dependency vulnerability scanning	npm audit in monthly Security Scan	JSON report	.github/workflows/security-scan.yml
IaC configuration scanning	Checkov across Terraform, App CI/CD, and Security Scan	SARIF + JSON reports	.github/workflows/terraform.yml
Git secret scanning	Gitleaks in PR Validation	JSON report	.github/workflows/pr-validation.yml
Approved registry enforcement	Kyverno require-approved-registry rule	YAML policy	policies/kyverno/image-verification.yaml
Image tag enforcement	Kyverno disallow-latest-tag rule	YAML policy	policies/kyverno/image-verification.yaml
Secret rotation	Monthly automated rotation with audit trail	SealedSecret YAML + rotation log	.github/workflows/secret-rotation.yml

The supply chain assessment demonstrates comprehensive coverage across vulnerability scanning (images, dependencies, IaC), provenance documentation (dual-format SBOM), source code protection (secret scanning), and registry governance (approved registries, tag enforcement). The dual-format SBOM generation (SPDX + CycloneDX) ensures compatibility with both government compliance requirements (SPDX is an ISO standard) and OWASP-aligned analysis tools (CycloneDX).

5.8 Compliance Metrics Summary

Table 18 consolidates the key compliance metrics across all evaluation dimensions.

Table 18: Compliance metrics summary

Metric	Value	Evidence
NIST SP 800-207 tenets satisfied	7/7	Table 12 mapping
Total policy rules (Kyverno + OPA + Falco)	39	Table 14 breakdown
CI/CD security tools integrated	10	Table 9 pipeline summary
CI/CD pipelines	6	.github/workflows/
Prometheus alert rules	22	-prometheus-rules.yaml
Grafana/Loki alert rules	14	Grafana alert configuration
MITRE ATT&CK techniques covered	13	Table 16 mapping
MITRE ATT&CK tactics covered	8/12	Table 16 analysis
SBOM formats generated	2 (SPDX + CycloneDX)	App CI/CD workflow
CRITICAL CVE deployment blocking	All tested cases were blocked	Trivy exit-code gate
Maximum runtime detection latency	< 30 seconds	Falco eBPF measurement
Secret rotation frequency	Monthly (automated)	Cron schedule in workflow
Audit trail storage	Git history + rotation logs	Immutable Git history

Table 19 summarises the policy and monitoring controls by type.

Table 19: Policy and monitoring control summary

Control Type	Count	Examples	Evidence
Kyverno validation rules	9	Non-root, no privileged, resource limits, drop capabilities, no latest tag, approved registry	policies/kyverno/.yaml
OPA deny/warn checks	16	Resource limits, probes, registry policy, hostPath denial, NetworkPolicy, service warnings	policies/opa/.rego
Custom Falco rules	13	Shell spawn, package management, crypto mining, reverse shell, sensitive file read, data transfer	clusters/test-cluster/05-infrastructure/falco.yaml
Prometheus alert rules	22	Service down, high CPU/memory, CrashLoopBackOff, OOM kill, Falco down, dropped events	clusters/test-cluster/05-infrastructure/rules.yaml
CI/CD workflows	6	PR validation, app CI/CD, Terraform, security scan, secret rotation, AI collector CI/CD	.github/workflows/.yaml

These results indicate that the framework achieves strong coverage across multiple security domains, including build-time vulnerability detection, deployment-time policy enforcement, and runtime threat monitoring. The combination of these controls ensures continuous compliance throughout the system lifecycle rather than relying on isolated or periodic checks.

5.9 Discussion of Results

The evaluation results demonstrate that continuous compliance can be achieved through the systematic integration of automated security controls across all six architectural layers. The key findings from the evaluation are:

Finding 1: Full NIST SP 800-207 tenet coverage is achievable with open-source tools.

The implementation achieves compliance with all seven NIST Zero Trust tenets using exclusively open-source security tools (Kyverno, OPA, Falco, Trivy, Checkov, Syft) deployed on open-source infrastructure (Kubernetes, Prometheus, Grafana, Loki). This demonstrates that Zero Trust compliance does not require expensive commercial security platforms.

Finding 2: The dual-engine policy approach provides complementary coverage.

Kyverno and OPA address different aspects of security policy. Kyverno's YAML-based rules are intuitive for Kubernetes operators and provide strong admission control. OPA's Rego language enables more complex cross-resource policies and extends naturally to non-Kubernetes contexts (e.g., validating Terraform configurations). The combination covers more security domains than either engine alone.

Finding 3: Left-shift security must be complemented by runtime detection. The CI/CD pipeline gates effectively prevent known vulnerabilities and policy violations from reaching production. However, runtime threats - shell injection, privilege escalation, data exfiltration, cryptocurrency mining - can only be detected during operation through kernel-level monitoring. The 39 policy rules span both deployment-time (25 Kyverno + OPA rules) and runtime (14 Falco rules), demonstrating the necessity of the defence-in-depth approach.

Finding 4: Automated secret rotation reduces credential exposure risk. Monthly rotation of MongoDB passwords, JWT signing keys, and API tokens - with automated sealing, GitOps deployment, and health verification - reduces the window of exposure for compromised credentials. The Git-based audit trail in `docs/rotation-logs/rotation-history.md` provides compliance evidence for each rotation event.

Finding 5: AI enrichment adds contextual value without introducing critical-path dependency. The AI Security Collector's parallel architecture ensures that alerting is never blocked by LLM unavailability or latency. The enriched reports provide investigators with threat assessments, investigation steps, and remediation recommendations that would otherwise require manual analysis, potentially reducing Mean Time to Understand (MTTU).

While the evaluation demonstrates the effectiveness of the framework in a controlled environment, it should be noted that real-world production systems may introduce additional complexity, including higher workload scale, diverse threat patterns, and integration with enterprise security operations.

6. Discussion

6.1 Critical Analysis

This study demonstrates that Zero Trust principles can be translated from a conceptual framework into a working, automated system for Kubernetes microservices. The implementation is comprehensive in its coverage across six layers, yet several critical observations merit discussion.

Strengths of the approach:

The framework's primary strength is its integration. Rather than deploying individual security tools as isolated point solutions, the project connects them into a continuous compliance pipeline where the output of one control feeds into the next. For example, Trivy scanning in the CI/CD pipeline produces SARIF reports that feed into GitHub Code Scanning; Falco events feed through Falcosidekick to Prometheus, Loki, PagerDuty, and the AI Collector simultaneously; secret rotation commits trigger Argo CD syncs that update running pods automatically. This integration means that a security event at any layer is visible across the observability stack.

The GitOps model provides a significant advantage for compliance. Because all infrastructure, application, and policy configurations are stored in Git, the repository serves as an immutable audit log. Any change - whether to a security policy, a Kubernetes manifest, or a secret rotation - is recorded with author, timestamp, and diff. This makes compliance verification a matter of Git history inspection rather than runtime interrogation, which is more reliable and reproducible.

Weaknesses and trade-offs:

The framework validates policies primarily in the CI/CD pipeline rather than through in-cluster admission webhooks. While the Kyverno policies are defined in enforce mode and are intended for cluster deployment, the current validation is primarily through the Kyverno CLI in the pipeline. This means that resources applied directly to the cluster (bypassing the CI/CD pipeline) would not be validated unless the in-cluster Kyverno admission controller is deployed. This is acknowledged as a scope limitation and is identified as future work.

The project does not include formal penetration testing or adversarial simulation. While the Falco rules are mapped to MITRE ATT&CK techniques and the CI/CD gates are verified through pipeline execution, the framework has not been subjected to red-team testing where an attacker actively attempts to bypass security controls. This limits the confidence in the framework's resistance to sophisticated adversaries.

The AI Security Collector depends on the Azure OpenAI API, introducing an external dependency. While the collector is designed to degrade gracefully (falling back to JSONL storage when Loki is unavailable, and skipping AI enrichment when the API is unreachable), the enrichment quality is ultimately dependent on the LLM's accuracy and relevance. Hallucination - where the model generates plausible but incorrect threat analysis - remains a risk that requires human oversight.

6.2 Research Question Answers

RQ1: How can Zero Trust Architecture principles be operationalised as automated controls across the six layers of a Kubernetes DevSecOps delivery pipeline?

This study demonstrates that Zero Trust principles can be operationalised through a layered approach where each architectural layer implements specific controls: infrastructure hardening via Terraform (Layer 1), Kubernetes security contexts and GitOps via Argo CD (Layer 2), application-level JWT authentication and RBAC (Layer 3), multi-pipeline CI/CD with ten security tools (Layer 4), eBPF-based runtime monitoring via Falco (Layer 5), and three-pillar observability with AI enrichment (Layer 6). The seven NIST SP 800-207 tenets are satisfied through the combination of these layered controls, as demonstrated in Table 12.

RQ2: What pipeline architecture balances comprehensive security verification with acceptable developer velocity and deployment frequency?

The six-pipeline architecture balances security and velocity through specialisation and parallelism. The PR Validation pipeline runs in under two minutes, providing fast feedback without impeding the developer workflow. The App CI/CD pipeline runs in eight to ten minutes, performing comprehensive scanning during the build phase while the developer has already moved on. Monthly scheduled scans and secret rotation run autonomously without developer involvement. The key architectural insight is that not all security checks

need to run on every commit - fast checks run on PRs, comprehensive checks run on merges, and audit checks run on schedules.

RQ3: How can policy-as-code enforcement be combined with runtime threat detection to achieve defence-in-depth for containerised workloads?

The dual-engine policy approach (Kyverno + OPA) provides deployment-time enforcement through twenty-five rules covering container privilege, resource management, image security, and network controls. Falco extends enforcement into runtime with fourteen rules covering execution, persistence, privilege escalation, credential access, and data exfiltration - threat categories that cannot be addressed through static policy validation. The result is a defence-in-depth model where thirty-nine rules span the complete lifecycle from admission to runtime, with each engine covering domains where it has the greatest detection capability.

RQ4: To what extent can AI enrichment of runtime security events reduce Mean Time to Understand (MTTU) for security incidents?

The AI Security Collector demonstrates that LLM-based enrichment can automatically generate threat assessments, investigation steps, and remediation recommendations for Falco security events, providing contextual analysis that would otherwise require manual analyst effort. The non-blocking architecture ensures that alerting latency is unaffected by AI processing time. While formal MTTU measurement is outside the current project scope (it would require a controlled experiment with SOC analysts), the enriched reports provide structured analysis that directly addresses the alert fatigue problem identified by Alahmadi *et al.* (2020).

6.3 Comparison with Existing Approaches

Table 20 compares This study's approach with existing Zero Trust and DevSecOps implementations in the literature.

Table 20: Comparison with existing approaches

Aspect	This study	NIST SP 800-207	DISA ZT Ref Arch	Shamim et al. (2020)	OWASP DevSecOps
Type	Implementation	Reference architecture	Reference architecture	Systematisation	Guideline
Scope	6 layers, full lifecycle	Conceptual tenets	7 pillars (conceptual)	11 considerations	Pipeline-focused
Policy engines	2 (Kyverno + OPA)	Not specified	Not specified	Discussed theoretically	OPA recommended
Runtime security	Falco eBPF (13 rules)	Continuous monitoring (tenet)	Discussed conceptually	Discussed theoretically	Not addressed
AI integration	GPT-4o-mini enrichment	Not addressed	Not addressed	Not addressed	Not addressed
SBOM	Dual format (SPDX + CycloneDX)	Recommended	Recommended	Not addressed	Recommended
Secret management	Automated rotation + audit	Mentioned	Required	Listed as consideration	Not detailed
Reproducibility	Open-source, Git-managed	N/A	N/A	N/A	Tool recommendations
Evaluation	NIST + MITRE ATT&CK mapping	N/A	N/A	Taxonomy only	N/A

The comparison highlights that This study provides a concrete, reproducible implementation that bridges the gap between conceptual frameworks (NIST, DISA) and practical guidance documents (OWASP) by demonstrating how Zero Trust principles translate into specific tools, configurations, and automated controls within a real Kubernetes deployment.

6.4 Limitations and Threats to Validity

Internal validity:

- The evaluation relies primarily on repository evidence and design analysis rather than runtime experiments with controlled attack scenarios. While this approach verifies that controls are implemented, it does not measure their effectiveness against real adversaries.
- The compliance mapping is based on the author's interpretation of NIST SP 800-207 tenets and MITRE ATT&CK techniques. Different interpretations could yield different compliance assessments.

External validity:

- The framework is implemented on a single, small-scale Kubernetes cluster (three nodes) with a simple microservices application. The scalability of the approach to larger, more complex environments has not been validated.
- The project uses Oracle Cloud Infrastructure; transferability to other cloud providers (AWS, Azure, GCP) would require modifications to the Terraform definitions but the Kubernetes-level controls should remain applicable.
- The FreshBonds application is a purpose-built demonstration system; real-world applications with more complex business logic may introduce additional security considerations.

Construct validity:

- Policy rule counts and tool integration counts are used as proxies for security coverage. A higher count does not necessarily equate to better security - rule quality, specificity, and false-positive rates are equally important but are harder to measure without operational data.
- The MITRE ATT&CK coverage analysis identifies which techniques are detectable but does not measure detection accuracy (true positive rate, false positive rate) under realistic attack conditions.

Specific limitations:

1. No formal penetration testing or red-team evaluation.
2. No multi-cluster federation or service mesh (mutual TLS) validation.

3. No quantitative performance benchmarking (pipeline execution times, Falco overhead).
4. No image signature verification (Sigstore/Cosign) implementation.
5. No unit or integration test suite for microservices.
6. NetworkPolicy implementation covers default-deny at the namespace level but lacks comprehensive per-service micro-segmentation.
7. In-cluster admission controller deployment (Kyverno as a webhook) is defined but not fully validated in production.

6.5 Implications for Practice

This study has several implications for practitioners seeking to implement Zero Trust in Kubernetes environments:

1. **Start with GitOps.** The Argo CD GitOps model provides the foundation for all other controls - it ensures that the cluster state is declarative, version-controlled, and auditable. Without GitOps, security configurations tend to drift from their intended state through manual changes.
2. **Use multiple policy engines.** No single policy engine covers all security domains. The combination of Kyverno (Kubernetes-native, YAML-based) and OPA (general-purpose, Rego-based) provides broader coverage than either alone. Practitioners should select engines based on their team's existing skill set and the specific policy requirements.
3. **Separate pipeline concerns.** Using six specialised pipelines rather than a single monolithic pipeline provides better isolation, clearer failure modes, and the ability to run different checks at different frequencies (every PR, every merge, monthly schedules).
4. **Automate secret rotation.** Manual secret rotation is error-prone and frequently deferred. Automated monthly rotation with health check verification and Git-based audit trails removes human bottlenecks and provides compliance evidence.
5. **Plan for runtime detection from the start.** Left-shift security is necessary but not sufficient. Runtime threats such as container compromise, privilege escalation, and data exfiltration require kernel-level monitoring (e.g., Falco with eBPF) that operates independently of the CI/CD pipeline.

7. Conclusion and Future Work

7.1 Conclusion

This study designed, implemented, and evaluated a comprehensive Zero-Trust DevSecOps framework for Kubernetes-based microservices, demonstrating that continuous compliance can be achieved through the systematic integration of automated security controls across the software delivery lifecycle.

The proposed framework is structured as a six-layer architecture encompassing infrastructure, platform, application, CI/CD, runtime security, and observability. It integrates policy-as-code enforcement, automated CI/CD security gates, runtime threat detection, secret lifecycle management, and AI-assisted security monitoring within a unified and reproducible system deployed on Oracle Cloud Infrastructure.

Evaluation against NIST SP 800-207 demonstrates alignment with all seven Zero Trust tenets, while MITRE ATT&CK mapping confirms coverage across multiple adversarial techniques. The results further show that the pipeline architecture effectively blocks deployments containing critical vulnerabilities while maintaining acceptable development performance, demonstrating that security enforcement can be achieved without significantly impacting delivery velocity.

The key finding of this research is that continuous compliance can be operationalised as a set of automated control points embedded across the delivery lifecycle, rather than relying on periodic manual audits. By integrating security verification at build time, enforcing policies at deployment, monitoring runtime behaviour, and enriching security events with contextual analysis, organisations can maintain a scalable and resilient security posture in cloud-native environments.

This research contributes a reproducible, end-to-end implementation that bridges the gap between Zero Trust theory and practical DevSecOps adoption in Kubernetes environments. It demonstrates how security controls can be unified, automated, and continuously enforced, providing both researchers and practitioners with a concrete reference model for implementing continuous compliance in modern software systems.

7.2 Future Work

While the proposed framework demonstrates strong capabilities in a controlled environment, several areas remain for future enhancement. First, the framework can be extended to support multi-cluster and multi-cloud deployments, enabling broader applicability in enterprise-scale environments. Second, integration with enterprise Security Operations Centre (SOC) workflows and automated incident response mechanisms would further improve operational effectiveness.

Additionally, the AI-assisted monitoring component can be enhanced through the use of domain-specific models and anomaly detection techniques to reduce false positives and improve contextual accuracy. Future work may also include large-scale performance evaluation under high workload conditions and formal penetration testing to assess resilience against advanced adversarial techniques.

Finally, the framework can be expanded to incorporate compliance mappings to additional regulatory standards, further strengthening its applicability in real-world enterprise and regulated environments.

Table 21 identifies areas for future improvement, categorised by priority and expected impact.

Table 21: Future work areas and planned improvements

#	Area	Description	Priority	Expected Impact
1	ML-based anomaly detection	Implement Isolation Forest, Autoencoder, and LSTM models for Falco event anomaly scoring, enabling detection of novel attack patterns beyond rule-based matching	High	Improved detection of zero-day threats
2	In-cluster admission control	Deploy Kyverno as a Kubernetes admission webhook for real-time enforcement at the API server level, beyond CI/CD-time validation	High	Prevents policy bypass through direct cluster access
3	Comprehensive NetworkPolicy	Implement per-service NetworkPolicy micro-segmentation beyond namespace-level default-deny	High	Stronger east-west traffic control
4	Image signature verification	Integrate Sigstore/Cosign for image signing in CI/CD and Kyverno verification policy for admission control	Medium	Ensures image provenance and integrity
5	Red-team evaluation	Conduct formal penetration testing against the deployed framework using adversarial simulation scripts mapped to MITRE ATT&CK techniques	Medium	Validates real-world detection effectiveness
6	Multi-cluster federation	Extend the framework to multi-cluster environments with centralised policy management and distributed monitoring	Medium	Validates scalability for enterprise adoption
7	Service mesh mutual TLS	Integrate Istio or Linkerd for automatic mutual TLS between services, eliminating the need for application-level TLS configuration	Low	Stronger internal communication security
8	Unit and integration testing	Develop comprehensive test suites for microservices using Jest and for ML models using pytest	Low	Improved code quality and regression detection

8. Contribution and Novelty

The primary contribution of this research lies in demonstrating how Zero Trust principles can be translated into an integrated, automated DevSecOps framework for Kubernetes. Unlike existing approaches that focus on individual security domains, this work provides a holistic, lifecycle-driven implementation with measurable outcomes, bridging the gap between theoretical security models and practical deployment.

Table 22 summarises the key contributions of This study.

Table 22: Project contributions summary

#	Contribution	Description
1	Zero-Trust DevSecOps reference implementation	A complete, reproducible, production-deployed implementation of Zero Trust principles for Kubernetes microservices, available as an open Git repository.
2	Lifecycle-based continuous compliance model	A model that connects PR validation, CI/CD scanning, GitOps deployment, scheduled audits, secret rotation, runtime detection, and incident reporting into a unified compliance pipeline.
3	Dual policy-as-code design	A complementary approach using Kyverno (Kubernetes-native YAML) and OPA (general-purpose Rego) that demonstrates how multiple policy engines can provide broader coverage than either alone.
4	Supply chain evidence model	Implementation of dual-format SBOM generation (SPDX + CycloneDX) with Trivy vulnerability scanning, approved registry enforcement, and image tag policy - producing machine-readable supply chain evidence at every build.
5	Secret lifecycle automation pattern	End-to-end automated secret rotation using GitHub Actions, Sealed Secrets, MongoDB Atlas API integration, and Git-based audit logging - demonstrating a complete credential lifecycle from generation through deployment to verification.
6	AI-assisted runtime security reporting	A non-blocking AI enrichment architecture that augments Falco security events with GPT-4o-mini-generated threat assessments, investigation steps, and remediation recommendations without introducing critical-path dependency.
7	MIS-level evaluation structure	A reproducible evaluation methodology mapping implementation evidence to NIST SP 800-207 tenets, MITRE ATT&CK techniques, and practical compliance metrics.

Novelty statement: The novelty of This study lies in the integrated artefact rather than in any single tool. Many projects use Trivy, Kyverno, Falco, or Argo CDs separately. This study combines them into a single continuous compliance pipeline with repository evidence, runtime monitoring, and AI-enriched incident context. The dual-engine policy approach, the non-blocking AI enrichment architecture, and the automated secret lifecycle management pattern are specific design contributions that have not been demonstrated in combination in the existing literature.

References

- Alahmadi, B. A., Axon, L. and Sherren, T. (2020) '99% false positives: a qualitative study of SOC analysts' perspectives on security alarms', *USENIX Security Symposium*, pp. 2203–2220.
- Anchore (2026) *Syft documentation*. Available at: <https://github.com/anchore/syft> (Accessed: 13 April 2026).
- Aqua Security (2024) *Trivy: a comprehensive and versatile security scanner*. Available at: <https://trivy.dev/> (Accessed: 13 April 2026).
- Biden, J. R. (2021) 'Executive Order 14028: Improving the Nation's Cybersecurity', *Federal Register*, 86(93), pp. 26633–26647.
- Bitnami Labs (2026) *Sealed Secrets*. Available at: <https://github.com/bitnami-labs/sealed-secrets> (Accessed: 13 April 2026).
- Bridgecrew (2024) *Checkov: policy-as-code for everyone*. Available at: <https://www.checkov.io/> (Accessed: 13 April 2026).
- Brown, S. (2018) *The C4 model for visualising software architecture*. Leanpub.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E. and Wilkes, J. (2016) 'Borg, Omega, and Kubernetes: lessons learned from three container-management systems over a decade', *ACM Queue*, 14(1), pp. 70–93.
- CIS (2024) *CIS Kubernetes Benchmark v1.8*. Center for Internet Security.
- Cloud Native Computing Foundation (2024) *CNCF Annual Survey 2024*. Available at: <https://www.cncf.io/reports/> (Accessed: 13 April 2026).
- Cloud Security Alliance (2022) *DevSecOps: integrating security into DevOps*. CSA Working Group Publication.
- DISA (2021) *Department of Defense Zero Trust Reference Architecture*. Defense Information Systems Agency.
- Falco (2024) *The Falco Project: cloud-native runtime security*. Available at: <https://falco.org/> (Accessed: 13 April 2026).
- Fitzgerald, B. and Stol, K. J. (2017) 'Continuous software engineering: a roadmap and agenda', *Journal of Systems and Software*, 123, pp. 176–189.
- Hevner, A. R., March, S. T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.
- Jeyaraj, A. (2023) 'Zero trust security in cloud-native environments: challenges and implementation strategies', *Journal of Cloud Computing*, 12(1), pp. 1–18.
- Kindervag, J. (2010) *Build security into your network's DNA: the Zero Trust network architecture*. Forrester Research Report.
- Kubernetes (2024) *Kubernetes documentation: Pod Security Standards*. Available at: <https://kubernetes.io/docs/concepts/security/pod-security-standards/> (Accessed: 13 April 2026).
- Kyverno (2024) *Kyverno: Kubernetes native policy management*. Available at: <https://kyverno.io/> (Accessed: 13 April 2026).

- Microsoft (2024) *Azure OpenAI Service documentation*. Available at: <https://learn.microsoft.com/azure/ai-services/openai/> (Accessed: 13 April 2026).
- MITRE (2024) *ATT&CK for Containers*. Available at: <https://attack.mitre.org/matrices/enterprise/containers/> (Accessed: 13 April 2026).
- Myrbakken, H. and Colomo-Palacios, R. (2017) 'DevSecOps: a multivocal literature review', in *Software Process Improvement and Capability Determination*. Cham: Springer, pp. 17–29.
- OPA (2024) *Open Policy Agent*. Available at: <https://www.openpolicyagent.org/> (Accessed: 13 April 2026).
- OWASP (2023) *OWASP DevSecOps guideline*. Available at: <https://owasp.org/www-project-devsecops-guideline/> (Accessed: 13 April 2026).
- PCI SSC (2022) *PCI DSS v4.0: Payment Card Industry Data Security Standard*. PCI Security Standards Council.
- Peffers, K., Tuunanen, T., Rothenberger, M. A. and Chatterjee, S. (2007) 'A design science research methodology for information systems research', *Journal of Management Information Systems*, 24(3), pp. 45–77.
- Red Hat (2024) *The State of Kubernetes Security Report 2024*. Available at: <https://www.redhat.com/en/resources/state-kubernetes-security-report> (Accessed: 13 April 2026).
- Rice, L. (2022) *Learning eBPF: programming the Linux kernel for enhanced observability, networking, and security*. O'Reilly Media.
- Rose, S., Borchert, O., Mitchell, S. and Connelly, S. (2020) *Zero Trust Architecture*. NIST Special Publication 800-207. Gaithersburg, MD: National Institute of Standards and Technology.
- Sanchez-Gordon, S. and Colomo-Palacios, R. (2020) 'Security as code: a systematic mapping study', in *International Conference on Computational Science and Its Applications*. Cham: Springer, pp. 159–174.
- Shamim, M. S., Bhuiyan, F. A. and Rahman, A. (2020) 'XI commandments of Kubernetes security: a systematization of knowledge related to Kubernetes security practices', *IEEE Secure Development Conference (SecDev)*, pp. 58–64.
- Sonatype (2023) *9th Annual State of the Software Supply Chain Report*. Sonatype Inc.
- Souppaya, M., Morello, J. and Scarfone, K. (2023) *Strategies for the integration of software supply chain security in DevSecOps CI/CD pipelines*. NIST Special Publication 800-204C. Gaithersburg, MD: National Institute of Standards and Technology.
- Souppaya, M., Scarfone, K. and Dodson, D. (2022) *Secure software development framework (SSDF) version 1.1: recommendations for mitigating the risk of software vulnerabilities*. NIST Special Publication 800-218. Gaithersburg, MD: National Institute of Standards and Technology.

Appendices

Appendix A: Security Tool Configuration Matrix

Table A.1: Complete security tool configuration matrix

Tool	Type	Phase	Pipeline(s)	Blocking	Output Format
Trivy	Container vulnerability scanner	Build, Post-deploy	app-cicd, ai-collector-cicd, security-scan	Yes (CRITICAL)	SARIF, JSON
Trivy (secret mode)	Embedded secret scanner	Build	app-cicd	Yes	JSON
Checkov	IaC security scanner	Build, Post-deploy	app-cicd, terraform, security-scan	Yes	SARIF, JSON
Gitleaks	Git secret scanner	Pre-merge	pr-validation	Yes	JSON
Syft	SBOM generator	Build	app-cicd, ai-collector-cicd	No (artefact)	SPDX, CycloneDX
OPA/Conftest	Policy validator (Rego)	Build, Post-deploy	app-cicd, security-scan	Yes	Text
Kyverno CLI	Policy validator (YAML)	Build, Post-deploy	app-cicd, security-scan	Yes	Text
kubeval	K8s manifest validator	Pre-merge	pr-validation	Yes	Text
npm audit	Dependency vulnerability scanner	Post-deploy	security-scan	Yes (MODERATE+)	JSON
kubeseal	Secret encryption	Rotation	secret-rotation	N/A	YAML
Falco	Runtime security monitor	Runtime	N/A (DaemonSet)	Alert	JSON
ESLint	Code linter	Pre-merge	pr-validation	Yes	Text
yamllint	YAML linter	Pre-merge	pr-validation	Yes	Text

Appendix B: Kyverno and OPA Policy Specifications

Table B.1: Kyverno ClusterPolicy - Pod security rules

Rule	Target Resource	Validation Pattern	Mode
run-as-non-root	Deployments (production, dev)	spec.template.spec.securityContext.runAsNonRoot: true	Enforce
disallow-privileged	All Deployments	spec.template.spec.containers[].securityContext.privileged: false	Enforce
require-resource-limits	All Deployments	containers[].resources.limits.cpu and .memory must exist	Enforce
drop-all-capabilities	All Deployments	containers[].securityContext.capabilities.drop: ["ALL"]	Enforce
disallow-privilege-escalation	All Deployments	containers[].securityContext.allowPrivilegeEscalation: false	Enforce
read-only-root-filesystem	Production Deployments	containers[].securityContext.readOnlyRootFilesystem: true	Enforce

Table B.2: Kyverno ClusterPolicy - Image verification rules

Rule	Target Resource	Validation Pattern	Mode
disallow-latest-tag	All Deployments	Image tag must not be :latest; explicit version required	Enforce
require-approved-registry	Production Deployments	Image registry must be docker.io, ghcr.io, or gcr.io	Enforce
require-image-pull-policy	All Deployments	imagePullPolicy must be Always or IfNotPresent	Enforce

Table B.3: OPA/Rego policy rules - security.rego

#	Rule Name	Severity	Policy Expression
1	runAsNonRoot	deny	Containers must set runAsNonRoot: true
2	privileged	deny	Containers must not set privileged: true
3	resource-limits	deny	Containers must define CPU and memory limits
4	readiness-probe	warn	Containers should define readiness probes
5	liveness-probe	warn	Containers should define liveness probes
6	privilege-escalation	deny	allowPrivilegeEscalation must be false
7	latest-tag	deny	Image tag must not be :latest
8	approved-registry	deny	Image registry must be in allowlist
9	hostPath-volume	deny	Pods must not use hostPath volumes
10	read-only-root	warn	Root filesystem should be read-only
11	dangerous-capabilities	deny	Must not add SYS_ADMIN, NET_ADMIN, ALL capabilities
12	drop-ALL	deny	Must drop ALL capabilities
13	env-var-validation	deny	Environment variables must not contain credential patterns

Table B.4: OPA/Rego policy rules - network.rego

#	Rule Name	Severity	Policy Expression
14	networkpolicy-required	deny	Every namespace must have at least one NetworkPolicy
15	loadbalancer-warning	warn	Services of type LoadBalancer should be reviewed
16	missing-selector	warn	Services without pod selectors should be flagged

Appendix C: Falco Custom Rules with MITRE ATT&CK Mapping

Table C.1: Complete Falco custom rule inventory with MITRE ATT&CK mapping

#	Rule Name	Priority	MITRE Tactic	MITRE Technique	Detection Condition
1	Shell Spawned in Container	WARNING	Execution	T1059	spawned_process and container and proc.name in (bash, sh, zsh, dash, ksh)
2	Package Management in Container	WARNING	Persistence	T1546	spawned_process and container and proc.name in (apt, apt-get, yum, pip, npm)
3	Detect Crypto Mining	CRITICAL	Impact	T1496	spawned_process and container and (proc.name in (xmrig, minerd) or proc.cmdline contains stratum)
4	Read Sensitive File	WARNING	Credential Access	T1552	open_read and container and sensitive_files
5	Privilege Escalation via Setuid	CRITICAL	Privilege Escalation	T1548	container and evt.type = setuid and evt.arg.uid = 0
6	Reverse Shell Detection	CRITICAL	Execution	T1059.004	container and fd.type = ipv4 and proc.name in (nc, ncet, bash, python)
7	Container Escape Attempt	CRITICAL	Privilege Escalation	T1611	container and proc.name in (nsenter, docker, runc, crictl)
8	Outbound Suspicious Port	WARNING	Command & Control	T1571	container and outbound and fd.sport in (4444, 5555, 6666, 1337)
9	Suspicious File Modification	CRITICAL	Persistence	T1543	container and open_write and fd.name in (/etc/passwd, /etc/shadow, crontab, authorized_keys)
10	Network Reconnaissance	WARNING	Discovery	T1046	container and spawned_process and proc.name in (nmap, masscan, zmap)
11	Suspicious DNS Query	WARNING	Command & Control	T1071.004	container and evt.type in (sendto, connect) and fd.name contains (tor, proxy, vpn)
12	Large Data Transfer	WARNING	Exfiltration	T1048	container and fd.type = ipv4 and evt.rawarg.res > 10485760
13	Read Sensitive File Untrusted	WARNING	Credential Access	T1552.001	open_read and sensitive_files and not trusted_containers

Table C.2: Falco sensitive_files macro definition

Macro	Files Monitored
sensitive_files	/etc/shadow, /etc/sudoers, /etc/pam.d/, ~/.ssh/, ~/.aws/, ~/.kube/

Appendix D: CI/CD Pipeline Specifications

Table D.1: CI/CD pipeline file specifications

Pipeline	Workflow File	Lines of YAML	Trigger	Estimated Duration	Jobs
PR Validation	pr-validation.yml	316	Pull requests to main/develop	< 2 min	code-quality, security-check, pr-size-check, summary
App CI/CD	app-cicd.yml	523	Push to main/develop, tags	8–10 min	detect-changes, policy-checks, build-and-scan, update-manifests, notify
AI Collector CI/CD	ai-collector-cicd.yml	474	Push to main (research/)	5–8 min	detect-changes, build-and-scan, update-manifests, notify
Terraform	terraform.yml	443	Push/PR to main (terraform/)	2–4 min	validate, security-scan, plan, apply, summary
Security Scan	security-scan.yml	610	Monthly (1st @ 2 AM UTC)	15–20 min	scan-images, scan-policies, scan-dependencies, scan-iac, summary
Secret Rotation	secret-rotation.yml	471	Monthly (1st @ 3 AM UTC)	3–5 min	rotate-secrets
Total	~2,837				

Appendix E: Grafana Alert Rule Summary

Table E.1: Grafana Loki alert rules by category

Category	# Rules	Alert Examples
Application Alerts	5	Error rate high, crash detection (CrashLoopBackOff), MongoDB connection failures, 5xx HTTP errors, payment processing failures
Log Stream Alerts	5	High error rate in pod logs, MongoDB disconnection pattern, payment service unavailability, auth failure rate spike, pod restart loop detection
Login Security Alerts	4	Brute force detection (failed login spike), single-user targeting (multiple attempts on one account), suspicious IP patterns, email enumeration attempts
Total	14	

Table E.2: Prometheus alert rules by category

Category	# Rules	Alert Examples
Infrastructure	6	DiskSpaceWarning (80%), DiskSpaceCritical (95%), NodeMemoryPressure, NodeDiskPressure, PVCAlmostFull (90%), NodeNotReady
Application	10	ServiceDown, HighMemoryUsage (>90%), HighCPUUsage (>90%), PodCrashLooping, ContainerOOMKilled, ServiceReplicas, ServiceAvailability, MongoDBConnectionFailures, APIGatewayHighLatency, PaymentProcessingFailures
Falco Monitoring	3	FalcoDown (DaemonSet unavailable), FalcoHighEventRate (>1000 events/min), FalcoDroppedEvents (kernel event loss)
Prometheus Self-Monitoring	3	PrometheusConfigReloadFailed, PrometheusTSDBCompactionsFailed, PrometheusNotConnectedToAlertmanager
Total	22	

Appendix F: Repository Structure and Evidence

Table F.1: Repository directory structure with artefact counts

Directory	Contents	File Count	Purpose
src/api-gateway/	Express.js reverse proxy with security headers	~10	API Gateway microservice
src/user-service/	Express.js authentication with JWT and bcrypt	~12	User authentication service
src/product-service/	Express.js product CRUD with RBAC	~12	Product management service
src/frontend/	React 18 + Vite + TailwindCSS	~30	Frontend SPA
terraform/	OCI infrastructure definitions	12	Infrastructure as Code
clusters/test-cluster/	Kubernetes cluster manifests	22+	Platform configuration
apps/freshbonds/	Helm chart with values and templates	~8	Application deployment
policies/kyverno/	Kyverno ClusterPolicy YAML	2	Kubernetes-native policies
policies/opa/	OPA Rego policy files	2	General-purpose policies
.github/workflows/	GitHub Actions workflow files	6	CI/CD pipelines
bootstrap/	Argo CD bootstrap application	1	GitOps entry point
research/	AI Security Collector (Python/FastAPI)	~15	AI-augmented monitoring
docs/	Project documentation	50+	Technical documentation
scripts/	Automation and setup scripts	~10	Build, deploy, and maintenance
thesis/	Academic thesis and report documents	3	Research deliverables

Verification commands for evaluation evidence:

```
# Count Kyverno rules
grep -c "name:" policies/kyverno/*.yaml

# Count OPA rules
grep -c "deny\|warn" policies/opa/*.rego

# Count Falco custom rules
grep -c "rule:" clusters/test-cluster/05-infrastructure/falco.yaml

# Count Prometheus alert rules
grep -c "alert:" clusters/test-cluster/05-infrastructure/*rules*.yaml

# Count CI/CD workflow files
ls .github/workflows/*.yaml | wc -l

# Count total YAML lines in workflows
wc -l .github/workflows/*.yaml

# Verify SBOM generation in pipeline
grep -n "syft\|sbom\|spdx\|cyclonedx" .github/workflows/app-cicd.yaml

# Verify Trivy blocking gate
grep -n "exit-code\|CRITICAL" .github/workflows/app-cicd.yaml

# Check secret rotation audit log
cat docs/rotation-logs/rotation-history.md
```